Verified Detection and Prevention of Concurrency Anomalies in Multi-Agent Large Language Model Systems

Sajjad Khan

Abstract—Agents in a multi-agent LLM system share state through memory stores and tool registries, and an agent's step spans an inference call of unbounded latency between its read and its commit. We formalize four concurrency anomalies for this regime—stale generation, phantom tool, causal cascade, tool-effect reordering—and name the classical condition each ports: stale generation is a failed commit-time read validation, and we prove (TLAPS) that its absence implies value-level consistency with the serial execution in commit order, a conditional guarantee over faithful histories whose hypotheses the artifact checks on every committed record. Each anomaly has a verified detector, each point of the 2^4 exclusion lattice is realized by a checked runtime configuration, and along one chain $L_0 \subsetneq \cdots \subsetneq L_4$ the disciplines are verified in Verus, with the L_1, L_2 and L_4 bytes the ones measured, including the L_1 decision of the live study; every claim is labeled with its assurance level, from checked model to verified bytes. The evaluation is scored on outcomes, not flags. On an unmodified LangGraph runtime under live models, read-set validation removes every commit grounded in a superseded read at one re-spent generation per conflict, yet at a 150 ms race offset the booking still contradicts the final trip date in 34–51% and 7–12% of sessions across two runs, because the revision landed after the commit; sweeping the race offset from 0 to 1.5 s drives the predicate to 0 and the residue to 52%. That residue is not a concurrency anomaly—it occurs in correct sequential executions—and no commit-time discipline reaches it; we define the residue predicate on effects and their recovered bases, show that under an open-world store it can only be owed, and realize a basis ledger that registers the obligation atomically (TLC-checked at every lattice point; invariant verified in Verus). On the same live study, with the commit decision taken by the verified store, the ledger with a rebooking compensator ends with $0/100$ wrong bookings on three API families where read validation alone leaves 12–63. Per sound committed operation, snapshot isolation costs $+0.4\%$ to $+11.1\%$ in tokens and pessimistic locking up to $+97\%$. A census of 98 public LangGraph repositories finds the stale-generation precondition in no graph site, which we report as a negative prevalence result, while a released tool executor batches an order-dependent pipeline on $30/30$ turns of one backbone. We state every number's provenance and every gap between predicate and harm.

Index Terms—Multi-agent systems, large language models, memory consistency, isolation levels, formal verification, TLA+, Verus.

1 Introduction

A multi-agent LLM system shares state. One agent writes a plan that another reads; a tool registry is consulted at planning time and called at execution time; a supervisor retracts a decision that a worker has already acted on. Each agent's step has the shape of a transaction—read, compute, commit—with one difference from every setting in which transactions have been studied: the compute phase is a neural-inference call that takes seconds to minutes, during which the read set is neither locked nor cheaply re-validated. Deployed frameworks place this phase inside a pinned snapshot (LangGraph's superstep), a re-rendered context (Letta) or a message queue (AutoGen), and none of them states what guarantee the shared state carries across it.

1.1 A failure, constructed and then observed

Two agents share a trip record. A flight agent reads "trip date = 14 June," drafts a reservation over a model call, and books it. During that call a second agent, acting for the user, moves the trip to 21 June. The flight agent commits a 14 June booking beside a 21 June trip date. No component failed; the history is even serializable, in the order in which the flight agent runs to completion and the revision follows. What it is not is serializable *in the order in which its operations committed*: the revision committed first, and a flight agent executing after it would have read 21 June. Figure 1 draws the shape.

We ran this scenario on an unmodified LangGraph runtime under live models (Section 6.1). With the second agent started inside the first agent's generation window, the framework committed the stale booking in every session in which the revision landed inside the window—60% and 91% of sessions on two API models, the remainder landing after the commit—and read-set validation at commit removed every such firing. The same data record a second fact the predicate does not see: when the revision lands *after* the commit, the booking contradicts the final trip date just the same, and no isolation discipline can prevent it. We report both numbers throughout, because the distance between them is the distance between a consistency predicate and the harm it is meant to track. The rates themselves are properties of the race we opened (a 150 ms offset; swept from 0 to 1.5 s in Figure 3) and we attach no prevalence reading to them.

What this paper is not claiming. A census of 98 public LangGraph repositories (Section 6.8) finds the structural precondition of the failure above—a reader and a writer of one last-value channel scheduled together—in none of 407 graph sites. We take that at face value: public LangGraph code is mostly sequential, and four of five frameworks we probed close the window by construction.

S. Khan is an independent researcher (e-mail: sajjadanwar200@gmail.com).
Artifact: https://github.com/sajjadanwar0/mac-consistency (TLA+ models, Verus proofs, the reference runtime, every harness, and the committed traces behind every table; `reproduce.sh` re-derives each figure from committed data). A supplementary document (§§ A–S) holds raw tables and proofs omitted here for length.

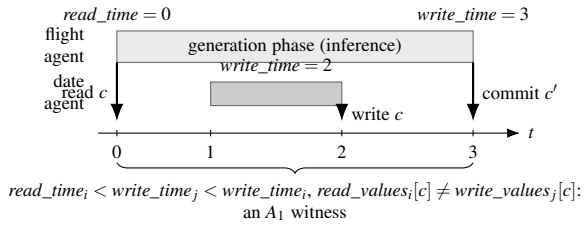


Fig. 1. The read–generate–commit window. Logical time is the length of the operation log. Operation i (flight agent) reads c at $read_time_i = 0$; operation j (date agent) commits a different value to c at $write_time_j = 2$, inside i's window; i commits at $write_time_i = 3$ grounded in the superseded read. The L_1 discipline re-validates the read set at $write_time_i$ and discards the generation.

The regime studied here is the one in which contention is designed in (parallel fan-out over a shared store, several agents acting for one user, a registry mutated while plans are in flight), and the contribution is not that the anomalies are common: it is a catalog with mechanized definitions, disciplines priced per unit of sound work, and an account of what each leaves behind.

1.2 Why existing theory does not operationalize these failures

Three traditions bear on the problem and none applies as it stands. Hardware models [5]–[8] presuppose bounded latency and static access sets; database isolation theory [9]–[11] supplies the anomaly-by-anomaly method we follow but holds the read set under lock or snapshot for the transaction's lifetime; replicated-storage consistency [12]–[15] treats operations as discrete events with summarizable signatures. The agent regime has unbounded generation latency, a tool registry that is first-class mutable state, and irreversible external effects—so the cascading-abort machinery of database recovery, which assumes effects can be undone, is exactly what fails.

1.3 Contributions

We claim no new concurrency theory. Each anomaly below is a classical one, and we say which. The contribution is the operational port of four of them to agent runtimes, its mechanization at stated assurance levels, and an evaluation that measures the gap between each predicate and the harm it tracks—including the part of the harm that is not a concurrency anomaly at all.

(C1) A catalog with classical names, and the residue it leaves (Section 3). Four history predicates in TLA$^+$—stale generation (A_1), phantom tool (A_2), causal cascade (A_3), tool-effect reordering (A_6)—each with a TLC witness and its classical counterpart: a failed backward validation [4], a dangling capability reference, recoverability with externalization as the commit point [21], and order preservation of external effects. Theorem 1 (TLAPS, 60 obligations) places A_1-freedom among the classical classes exactly, as a conditional guarantee whose hypotheses (per-agent non-overlap, read-before-commit, faithful records) are stated, checked on every committed record, and violated by one deployed construct we name. Beside the catalog we define the *residue predicate* R: an irreversible effect is out while ordinary later writes have revised the state it was derived from. R is not an anomaly—it fires in correct sequential executions—and we say so; it is the harm no commit-time discipline reaches, decided on the records by argument provenance, and on the live study A_1 and R partition the outcome column (Section 3.6).

(C2) A lattice, realized (Section 4). The 2^4 exclusion lattice is trivial; what is not is that every point is inhabited by a checked runtime configuration (64 TLC runs, no mismatch, bounded as stated), that one chain $L_0 \subsetneq \cdots \subsetneq L_4$ has a verified realizing runtime at every level, and that the basis ledger that answers R composes with every point without disturbing its verdict (160 further runs).

(C3) Verified detectors and disciplines (Section 5). 371 Verus obligations, no `assume` or `admit`, under two enumerated axioms and three enumerated stubs: detectors sound and complete against their specifications, two concurrent L_1 stores proved window-free under a verified reader–writer lock, L_2 (output commit), L_3 (commit-order sequencing) and L_4 (validation at dispatch) runtimes whose L_2 and L_4 bytes are the ones measured. Table 2 labels every claim with its assurance level (checked model, verified decision procedure, verified bytes, unverified port, open) and names each correspondence between TLA$^+$ definition, Verus restatement, executed code and scorer, with its hypothesis and the gate that checks it on the data. We account for the obligations by what they bear (supplement § S): 236 underpin a deployed or measured artifact and 135 are scaffolding. The basis ledger that answers R is verified (7 obligations) in Verus: its invariant is preserved by every step and excludes the residue witness; its executable form in the live study is harness code, labeled as such.

(C4) An evaluation scored on outcomes, not flags (Section 6). For each discipline we report the anomaly it excludes, the harm that remains, and the price: on the live LangGraph study, the L_1 residue is R in 34% and 7% of sessions at the 150 ms offset (51% and 12% in a second run), decided on the records; with the commit decision taken by the verified L_1 store through a 100-line bridge the residue is the same kind of number ($A_1 = 0/300$; booking $\neq$ final 2, 51, 52 of 100 on haiku, gpt-4o-mini, deepseek-chat), and the basis ledger with a compensator removes it (booking $\neq$ final $0/100$ on all three families (5, 45, 53 obligations, all discharged)) at one compensation generation per supersession; on 1,000 seeded cascades per depth, output commit removes exposure exactly when the review precedes the release and refuses the retraction otherwise; per sound committed operation, snapshot isolation costs $+0.4$–11.1% and pessimistic locking $+1.2$–97% in tokens, with a wall-clock measurement whose run-order confound we quantify rather than assume away.

(C5) Where the preconditions occur (Section 6.8). A census of 98 public LangGraph repositories (407 graph sites) finds no co-scheduled reader and writer of a last-value channel—a negative result for A_1's structural precondition, which we report as one—while a released tool executor dispatches a model-emitted, order-dependent pipeline concurrently on $30/30$ turns of one backbone and never on another, which is the case for A_6.

What this paper does not establish. It does not establish a production prevalence for any anomaly: every occurrence figure is a race our harness opened or a rate on our own workloads, and the census measures how public code is written, not how often it runs. It does not verify the instrumentation or the Python harnesses (trust base: Section 5.7). And a verified arm's zero is a theorem executing, not evidence, so the evaluation leads with costs and with counts that read no runtime flag.

2 Runtime Model

A system consists of a finite set of agents A, a shared memory M mapping cells to values with a distinguished sentinel NULL, and a

shared registry R of available tools. An agent executes a sequence of *operations*, each in three phases: a *read* phase that captures the values of a chosen subset of cells (the operation's *read_set*) together with the registry contents (*read_registry*), stamping the read with the current log length as *read_time*; a *generation* phase, dominated by inference latency and not treated as instantaneous; and a *write* phase that atomically appends a record to the log, installing the planned writes (*write_set*, *write_values*), the selected tool (*planned_tool*) and the registry at commit (*write_registry*), stamped *write_time*. The log length is a logical clock [18]; a history h is the log, $h \in \mathrm{Seq}(\mathit{OpRecord})$. The specification is `Memory.tla`.

Store axiom. A read returns the latest committed value at or before its read time. TLC confirms this and the two hypotheses below as invariants of `Memory.tla` over all 2,961,434 reachable states at the checked bounds (Section 3.1).

Per-agent non-overlap; read before commit. Two operations of one agent do not overlap in $[\mathit{read_time}, \mathit{write_time}]$; every operation reads strictly before it commits. Both are invariants of the model, and both are load-bearing for Theorem 1 and for the detector transcription of Section 5.1. A framework that runs one node as several parallel instances under one name (LangGraph's `Send` fan-out) violates the first and is outside the model; Section 6.8 counts how often that shape appears in public code.

Determinism is not assumed by the theorems. Every mechanized result ranges over recorded histories with write values free. Under sampling, a value mismatch need not change the downstream decision, so the A_1 detector is a sound candidate screen that over-reports by a factor Section 6.7 measures. Durable-execution engines take the same stance [52]: replay is deterministic because every non-deterministic step's result is recorded, not because the step is.

Atomic commit is at the record level. External effects are outside the store and are governed separately: their order by A_6 and L_3, their release relative to a still-retractable basis by A_3 and L_2. At-most-once externalization is assumed and not modeled; Section 8 reports a deployed resumption path that violates it.

No replication. The memory is one logical store. Split view under replication (A_4) is formalized separately at the Verus model level (supplement § J) and plays no role in the lattice.

Placement. The model does not derive from any one runtime. LangGraph is driven through it live (Section 6.1); three others are placed informally (supplement § D).

3 THE ANOMALY CATALOG

Four anomaly predicates over histories are formalized in TLA$^+$ (`Anomalies.tla`); each has a TLC witness at $|A|{=}2$, $|\mathit{Cells}| \le 2$, $\mathit{MaxOps} \le 4$, reproduced by the artifact's `reproduce.sh` and reproduced independently for this version. Table 1 names each predicate's classical counterpart and the regime feature that forces the port. The numbering (A_1,A_2,A_3,A_6) is kept from the public artifact; an earlier A_5 was subsumed by A_1. Section 3.6 then defines, outside the catalog, the predicate for what the four leave behind.

3.1 A_1: stale generation

Definition 1 (Stale generation)**.** A history h exhibits *stale generation* when there exist $i \neq j$ with $h[i].\mathit{agent} \neq h[j].\mathit{agent}$ and $c \in h[i].\mathit{read_set} \cap h[j].\mathit{write_set}$ such that

$$h[i].\mathit{read_time} < h[j].\mathit{write_time} < h[i].\mathit{write_time}$$
$$\text{and} \quad h[i].\mathit{read_values}[c] \neq h[j].\mathit{write_values}[c].$$

TABLE 1
Each cataloged predicate, its classical original, and what the agent regime changes. A_4 (split view) needs replication and is outside the single-store model.

Predicate	Classical condition	What the port changes
A_1 stale generation	failed backward validation [4]; its absence is commit-order read consistency (Theorem 1)	the read phase is an inference call: unbounded, unlocked, not cheaply re-validated
A_2 phantom tool	dangling capability reference [20]	the registry is mutable state with no relational analogue; the call reaches whatever is live at dispatch
A_3 causal cascade	recoverability, externalization in the role of commit [21]	effects are irreversible, so the dependent must be *held*, not aborted after the fact
A_6 effect reordering	order preservation of external effects (2PC, write buffering)	a batch the model declared parallelizable is dispatched concurrently; completion order is the commit order
R (residue, Section 3.6; not an anomaly)	view maintenance [107], with the view outside the system; why-provenance [105] for the basis	the effect is irreversible and its basis is revisable, so the window after commit never closes; only an obligation can be guaranteed (Proposition 1)

Witness. TLC finds a five-state counterexample: a_2 reads $c_1 =$ NULL at $\mathit{read_time} = 0$; a_1 commits $c_1 = v_1$ at $\mathit{write_time} = 1$; a_2 commits at $\mathit{write_time} = 2$ with $\mathit{read_values}[c_1] =$ NULL.

Classical placement. The minimal witness is a single read–write antidependency from i to j, serializable in the order i; j, and none of Adya's proscribed phenomena [10]; TLC exhibits such a history (claim `A1OnlyOnNonSerializable`, violated as expected). A_1 is therefore not an isolation anomaly in the classical sense. Its *absence* is nevertheless a classical correctness class, and we state which one at the level at which the theorem holds.

Definition 2 (Commit-order read consistency)**.** A history is *commit-order read consistent* when every operation read, for every cell in its read set, exactly the value it would have read had it executed atomically at its own commit point: for all i and $c \in h[i].\mathit{read_set}$, $h[i].\mathit{read_values}[c]$ is the value of the latest write to c with $\mathit{write_time} < h[i].\mathit{write_time}$, or NULL if there is none.

Writes are installed in commit order in any case, so Definition 2 is value-level equivalence, on the reads, to the serial execution in commit order. It is the value-level analogue of Weikum and Vossen's commit-order-preserving serializability [3] and not that condition itself, which is conflict-based; under value-unique writes the two coincide on reads (view equivalence [29]).

Theorem 1 (A_1-freedom implies commit-order read consistency)**.** *Let h satisfy the store axiom, read-before-commit and per-agent non-overlap. If h has no A_1 witness, then h is commit-order read consistent.*

The proof (`CommitOrder.tla`; TLAPS, 60 obligations, no omitted step) is Kung and Robinson's backward-validation argument run against our definitions: if some operation's read of c disagrees with the latest write to c before its commit, that write lies inside the operation's window, was made by another agent (one agent's operations do not overlap), and wrote a different value—an A_1 witness. Fifteen TLC verdicts (`check_commitorder.sh`) fix what the statement says, and all fifteen reproduce: over all 35,659 histories of length at most three the implication holds; it is not vacuous; it fails when either droppable hypothesis is removed; the relational definitions equal `Memory.tla`'s and a literal serial

replay; the three hypotheses are invariants of the operational model over 2,961,434 reachable states; and the converse fails in exactly one shape—a value overwritten and then restored inside the window fires A_1 although the history is commit-order read consistent. The predicates nest strictly:

$$\neg\text{CORC} \implies A_1 \implies \text{validation fails.}$$

Reach. The theorem is a conditional guarantee over faithful histories that satisfy its three hypotheses; it says nothing about serializability of an application, about effects, about reads the instrumentation did not declare, or about a runtime that violates per-agent non-overlap. One deployed construct does: LangGraph's `Send` fan-out runs several instances under one agent name and is outside the model (Section 2). On the committed data both droppable hypotheses hold—0 same-agent overlapping windows and 0 reads not before their commit in 36,036 records over 10,488 traces (`hypothesis_gates.py`, a gate that fails on the first violation)—and a runtime can enforce the first by refusing a second operation of an agent whose window is open. The converse shape is a false positive of L_1, priced in discarded generations; on the live records it occurs in 0 of 710 A_1 witness pairs. Two corollaries are mechanized with the theorem. The classical lost update (P4 of Berenson et al. [9]) is an A_1 witness in which the stale reader also writes the cell it read, so L_1 excludes it. A *blind* overwrite—a write by an operation that never read the cell—is not an anomaly in isolation theory and is outside the catalog; it matters in Section 6.8, because the one in-the-wild bug we reproduce is of that kind, and in Section 6.1, because a revision that lands *after* the reader's commit is of that kind too.

3.2 A_2: phantom tool

Definition 3 (Phantom tool)**.** h exhibits a *phantom tool* when for some i,

$$h[i].\mathit{planned_tool} \neq \text{NULL},$$
$$h[i].\mathit{planned_tool} \in h[i].\mathit{read_registry},$$
$$h[i].\mathit{planned_tool} \notin h[i].\mathit{write_registry}.$$

An agent selects a tool from the registry it read; the registry mutates during generation; the commit emits a call to a capability that no longer exists. TLC's witness has four states. *Tool validity*, canonically, is membership of the (identifier, signature) pair: a tool is the same tool while both are unchanged. Definition 3 is stated over identifiers, the TLA$^+$ registry being a set of them; the Verus restatement of Section 5.4 maps identifiers to signatures and fires additionally when a tool is re-signed under its old name. The two coincide under the encoding the executable dispatcher uses, in which re-signing a tool is removal of the old pair and insertion of a fresh one, so that a re-signed tool is a removed tool to Definition 3; the scorer and the L_4 dispatcher decide the pair form, and Table 2 records the correspondence as an encoding, not a proof. A tool the agent names that was never in *read_registry* (a hallucinated call) is not A_2 and not in the catalog.

3.3 A_3: causal cascade

When an operation is retracted in a compensating (saga-style) runtime, any operation that read a value it produced is grounded on a basis no surviving commit supports. The classical remedy, cascading abort, flags the dependent aborted. In the agent regime that remedy fails in a specific way: if the dependent's tool effects have already been issued, the flag changes nothing that reached the world. The cataloged predicate is therefore stated over externalization.

Definition 4 (Causal cascade)**.** Let each record carry *aborted* and *externalized* flags and a predecessor set *preds* (the operations whose committed writes it observed, transitively closed). h exhibits a *causal cascade* when for some j, p:

$$h[j].\mathit{externalized} \wedge p \in h[j].\mathit{preds} \wedge h[p].\mathit{aborted}.$$

The surviving-dependent form, $\neg h[j].\mathit{aborted} \wedge p \in h[j].\mathit{preds} \wedge h[p].\mathit{aborted}$, is kept beside it as CASCADEUNPROPAGATED; a cascading abort falsifies that form by flagging j after j's effects are out, and Section 6.3 measures the separation on runnable arms.

Prevention is output commit, and output commit has a residue. Under Definition 4 a runtime must hold an operation's effects until every operation in its closure has released its own, and must refuse to retract an operation whose effects are out. The first clause is prevention. The second keeps the predicate false by refusing to set the flag it reads, and we do not count that as prevention: a refused retraction is a basis someone wished to withdraw after release, the ordinary rule of recoverability that a committed transaction cannot abort, and every measurement in Section 6.3 counts its residue beside the zero.

What the detector decides. A trace without provenance fields cannot decide Definition 4. What it can decide is an *unsupported read*—a committed read of a value that no surviving committed write produced at or before the read. The verified `detect_a3` decides exactly that screen, and its relation to Definition 4 is five mechanized facts (supplement § G): it is not sound in general (a value collision hides the aborted writer), it is sound under value-unique writes, the hypothesis black-box isolation checkers also need [42], [43], it holds along chains, it is not complete, and it is blind to relabeled effects. We therefore claim Definition 4 only where the hypothesis holds, and check it: value-unique writes hold by construction in the synthetic cascades (distinct value ranges per writer) and in all 600 live replay sessions (0 collisions), the two data sets the detector scores; they fail in 126 cells of the pilot and occurrence-study traces, where only A_1 is scored. A recorder that stamps each written value with its writer's operation identifier makes the hypothesis a system invariant.

3.4 A_6: tool-effect reordering

Each record additionally carries an issuance order *io* and a commit order *co* over its external effects; *write_set* and *write_values* derive from *co*.

Definition 5 (Tool-effect reordering)**.** h exhibits *tool-effect reordering* when for some i,

$$|h[i].\mathit{io}| \geq 2, \quad h[i].\mathit{co} \neq h[i].\mathit{io},$$
$$h[i].\mathit{co} \text{ is a permutation of } h[i].\mathit{io}.$$

The minimal witness (three states, one agent, two cells) is benign over store cells, where writes to distinct cells commute; the predicate names an anomaly only under the reading in which each element of *io* is an external effect whose order an observer can see. It over-reports on commuting effects and, because it requires a permutation, is silent on dropped and duplicated effects; Section 8 exhibits the duplicated case in a deployed framework.

3.5 Snapshot insufficiency

L_1 is not a snapshot guarantee: "reads reflect memory at read time" is the store axiom of Theorem 1, holds on every reachable log, and TLC exhibits a history satisfying it on which A_1 fires (`MC_A1_struct`; supplement § B). The live instance is a runtime that pins a consistent snapshot per node—LangGraph's superstep—and has not thereby obtained L_1 (Section 6.1).

3.6 What the catalog leaves: the residue predicate

Every predicate above is decided at or before an operation's commit, and a discipline that excludes all four still leaves a harm the live study measures in a third of sessions (Section 6.1): the external effect a commit released is irreversible, while the state it was derived from is ordinary revisable state that any later operation may overwrite. This is not a concurrency anomaly. It fires in correct sequential executions—book against the date known at commit, then be contradicted by a legitimate revision—and classical isolation has no name for it because a committed transaction's inputs are final by definition; the nearest notions are view maintenance [107] with the view outside the system, and the saga's compensation [22]. We define it because an evaluation scored on outcomes needs a predicate for the outcome's other half, and keep it outside the catalog and the lattice.

Each operation record carries a *basis*, the (cell, value) pairs its external effect was derived from, and an *obligation* flag. The basis is recovered by *argument provenance*: a read value that appears verbatim among the effect's arguments is part of its basis, and nothing is declared by the agent. This is why-provenance [105] restricted to copying, with that tradition's known limit: a value that reaches the effect through a computation leaves no basis, so the extractor under-reaches and never over-reports. In the model the basis is a nondeterministic subset of the read values, so every extractor is covered.

Definition 6 (Residue predicate)**.** *h exhibits a superseded-basis effect (R; `NoA7` in the artifact) when for some i, $h[i].externalized$, $\neg h[i].obligation$, and for some $(c,v) \in h[i].basis$ the latest write to c committed after $h[i].write_time$ wrote a value other than v:*

$$\exists j:\ c \in h[j].write_set \ \wedge\ h[j].write_time > h[i].write_time \\ \wedge\ h[j].write_values[c] \neq v \\ \wedge\ \forall k:\ (c \in h[k].write_set \wedge h[k].write_time > h[i].write_time) \\ \Rightarrow h[k].write_time \leq h[j].write_time.$$

A_1 and R partition "an effect is out on a basis the state no longer holds" by whether the superseding write committed inside the operation's window or after its commit; a value overwritten and then restored fires neither, and R carries no agent conjunct because the obligation is owed whoever supersedes. On the live data the partition is exact (Table 3): $A_1 + R = 100$ in every unguarded cell, and under L_1 the A_1 column is empty and the residue is entirely R. The two sides of that identity are computed independently—the outcome column compares the booking with the final store state and never reads a basis; R reads the trace's write order and never the final state—so the identity is a check on the extractor, not a tautology of it; on this workload the extractor recovers the basis of 4,500/4,500 bookings because the booking tool takes the date as a structured argument, and its blind spot is stated in Section 8. TLC's witness has five states and is not an A_1 witness: an operation reads $c_1 = \text{NULL}$ and commits an effect with basis $\{(c_1, \text{NULL})\}$, released at commit; a later operation commits $c_1 = v_1$.

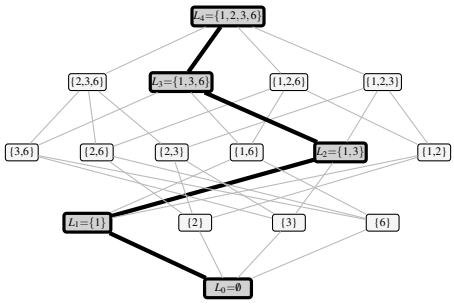


Fig. 2. The lattice $\langle 2^{\{A_1,A_2,A_3,A_6\}}, \subseteq \rangle$ (sets abbreviated by index) and the chain realized by verified runtimes (bold). Every point is realized by the parameterized runtime of Section 4.2, with or without the basis ledger.

Proposition 1 (What remains after release)**.** *Let D be any discipline under which an effect e with basis B is released at a finite time T, and let the store accept, at any time after T, a write to a cell of B by an operation that does not depend on e (the open-world hypothesis). (i) There is an execution of D in which a write committed after T supersedes B while e is out. (ii) If obligations are never cleared, $\neg R$ holds in every reachable state of D if and only if, in every execution, e's obligation is set in the same step as, or before, the step that completes the other conjuncts of Definition 6—the superseding commit for a released effect, the release for an effect superseded while held.*

Both halves are immediate—(i) because the write is admitted after a release that cannot be recalled, (ii) because Definition 6 is "superseded and not owed" and a monotone flag set by that step stays set—and we record them as a proposition, not a theorem, because their content is to fix what a discipline can promise about R under the open-world hypothesis: not a current basis, but an obligation registered atomically with the step that completes the witness, which the ledger of Section 5.5 does at both steps. Dropping the hypothesis is possible and is not free: freezing the cells of B until the effect is acknowledged is a lock held past commit, with the pessimistic column of Table 6 as its price floor; output commit (Section 3.3) holds effects only until their *causal* basis is externalized, a finite condition. What a registered obligation buys in the world—a rebooking, a cancellation, a human notification—is a property of the compensator and is measured, not proved (Section 6.1).

4 The Lattice and One Chain Through It

A consistency model is a predicate over histories, characterized by the set of anomalies it excludes [10]. With $\mathscr{A} = \{A_1, A_2, A_3, A_6\}$ the design space is the Boolean lattice $\langle 2^{\mathscr{A}}, \subseteq \rangle$: sixteen points, bottom $\emptyset$ admitting everything, top $\mathscr{A}$ admitting nothing (Figure 2). We name five points on one maximal chain,

$$L_0 \equiv \emptyset,\ L_1 \equiv \{A_1\},\ L_2 \equiv \{A_1, A_3\},\ L_3 \equiv \{A_1, A_3, A_6\},\ L_4 \equiv \mathscr{A},$$

chosen on operational grounds (A_1 first because it is the basic read–generate–commit failure; A_2 last because registry stability across the operation lifetime is the costliest single requirement); the other twenty-three linearizations are equally valid and the lattice, not the chain, is primary. The residue predicate R is not a generator: it is not an anomaly, and its answer, the basis ledger ℓ of Section 5.5, composes with any point.

4.1 What is and is not a result

The chain's inclusions are definitional and its strictness needs one witness per step; neither is a result. What requires work is *realizability*: a runtime configuration that excludes exactly each point's anomalies, and for the five named points a verified realizing runtime—established here by bounded model checking, and claimed at the bounds stated.

4.2 All sixteen points, mechanically; the ledger composes

`Guarded.tla` extends the runtime with four commit-time disciplines, each enabled by a Boolean constant and each acting on its own carrier: G_1, read-set validation (an operation whose read set was overwritten after its read may not commit); G_2, registry validation (a planned tool no longer in the registry may not be dispatched); G_3, causal tracking with output commit (an operation records the closure of committed writers whose values it read, may not commit while any is aborted, externalizes only once its closure has externalized, and once externalized is irrevocable; a retraction cascades to every unexternalized dependent); G_6, commit-order sequencing (effects externalize in issuance order). The carriers are disjoint—read set and write times; registry and planned tool; closure and abort flags; issuance and commit orders—so enabling one guard neither creates nor removes another anomaly, and the matrix confirms it on every combination.

For each of the sixteen guard sets S and each anomaly a, TLC checks $R(S) \models \neg a$: for $a \in S$ the invariant must hold exhaustively, for $a \notin S$ it must be violated. All 64 runs match (`lattice16.sh`; $|A|$=2, $|Cells|$=2, $|Values|$=1, $MaxOps$=2, symmetry on agents and cells; the 32 safety runs are exhaustive over 380,920–2,015,279 distinct states). Because CAUSALCASCADE ranges over flags and predecessor sets and never over cells, the sixteen NOA3 points are rechecked at $MaxOps$=3 with one cell, where a transitive cascade first becomes reachable: 16/16 match. A single-value domain makes Definition 1's value conjunct vacuous on non-null reads; the safety half re-run at $|Values|$=2 stays safe in all 32 cells, and a targeted check shows the non-null-read witness class needs both a second value *and* a third operation (supplement § L); the two-value safety half is re-run on the current model (all 32 safety runs SAFE, 2.4–17.3M states). Every point is thus inhabited by a runtime that prevents exactly its avoidance set, at these bounds; the Verus product of the L_2, L_3 and L_4 models (Section 5.6) mechanizes the top point without a bound.

The ledger composes. `Guarded5.tla` adds a fifth constant G_7, the basis ledger of Section 5.5, which refuses nothing and writes only the basis and obligation fields. `lattice32.sh` runs all 160 combinations of the five constants against the four anomalies and R at the same bounds. The 64 anomaly cells with G_7 on return the verdicts of `lattice16.sh` one for one, so the ledger disturbs no point of the lattice; R is reached without G_7 in a five-state trace at every point and excluded with G_7 at every point (exhaustive; 5,213,012 distinct states at the bottom point, 9,688,601 with output commit, where a held effect can be superseded before its release). All 160 runs match their expectation (0 mismatches); the exhaustive runs range over 1,803,848–9,688,601 distinct states, and the sixty-four commit-time cells with G_7 off return the verdicts of `lattice16.sh` one for one.

5 MECHANIZATION

Three tools carry the development, at assurance levels Table 2 labels claim by claim. TLC checks the witnesses, the lattice and ledger matrices and the fifteen Theorem 1 verdicts (bounded, exhaustive within bounds). TLAPS proves Theorem 1 (60 obligations), a coherence check of the chain definitions (`Hierarchy.tla`, 15) and a same-agent exclusion lemma (`A1LowerBound.tla`, 28); none is a substantive result except the first. Verus [24] carries the substantive mechanization: 371 obligations (392 counting four helper files), zero errors, no `assume` or `admit` in any proof body, regenerated by `verus_count.sh` against a pinned build (`verus-version.txt`; a newer verifier makes proof files stop compiling rather than fail, which the counter reports as such). Of these, 236 bear a deployed or measured artifact's safety claim and 135 are scaffolding (superseded refinements, the A_4 model, the model-only Verus mirror of `Guarded.tla`), verified but not load-bearing; supplement § S inventories every file.

5.1 Verified detectors, and one transcription gap

The four detectors are verified sound *and* complete against Verus `spec` restatements of the TLA$^+$ predicates (24 obligations). `detect_a1` is a two-sided equivalence whose completeness half strengthens each of three nested loop invariants to carry "no witness in the prefix searched." `detect_a2` and `detect_a6` are single-record equivalences; `detect_a3` is exact for the flat-trace residue of Section 3.3.

The restatement is hand-written and checked by inspection, and inspection found a gap we state rather than hide. The Verus `OpRecord` carries no agent field, so its A_1 predicate omits the different-agent conjunct of Definition 1; and it decides "no witness" when a read value is absent, where the TLA$^+$ compares against NULL. Neither matters on the histories the model admits, and we say why.

Lemma 1 (Same-agent exclusion)**.** *Under per-agent non-overlap and read-before-commit, if* $c \in h[i].read_set \cap h[k].write_set$ *and* $h[i].read_time < h[k].write_time < h[i].write_time$, *then* $h[i].agent \neq h[k].agent$.

The lemma is step $\langle 2 \rangle 2$ of the TLAPS proof of Theorem 1 and the content of `A1LowerBound.tla`. It makes the Verus predicate and Definition 1 coincide on every history satisfying the model's invariants; on a history that violates non-overlap the Verus predicate is strictly stronger—it over-reports, the safe direction for a detector. The instrumentation records a value for every read cell, so the absent-value case does not arise: across all 36,036 committed operation records there is no read cell without a value and 0 same-agent overlapping windows (`hypothesis_gates.py -check`). The published tables were scored by `rust-analyser`, a hand-written port of the verified detector; a gate (`predicate_matrix_a1.py`) scores every committed cell under the TLA$^+$, the Verus and the port's transcription and fails on any divergence, and all three agree on every cell. Wiring the verified `detect_a1` into the scorer is a build step the artifact's delegation shim prepares and this version has not taken.

5.2 L_1: two concurrent stores whose lock is verified, not assumed

Three abstract state machines—pessimistic locking, serializable snapshot isolation (SSI), and default snapshot isolation with its no-write bypass—are proved A_1-free under sequential interleaving (40 obligations; default-SI conditionally, on workloads with no read-only commit, which is the residual

TABLE 2
Every claim at its assurance level. *Model*: checked by TLC within bounds; *proof*: TLAPS or Verus, no `assume`/`admit`; *bytes*: the verified code is the code measured; *port*: unverified code gated against a verified artifact on the committed data; *open*: stated, not established. Correspondences name the hypothesis under which two formalizations agree and the gate (`harness/macaudit/`) that checks it.

Claim	Definition	Proof / check	Executed by	Correspondence, hypothesis, gate; status
Anomaly witnesses, lattice realizability	TLA$^+$	TLC, $\|A\|$=2, $\leq$ 2 cells, $\leq$ 2 values, MaxOps $\leq$ 4 (witnesses), $\leq$ 3 (matrix)	—	**model**: bounded; 64+16+32 two-value safety runs
Theorem 1	TLA$^+$	TLAPS 60	—	**proof**; hypotheses H1–H3 are model invariants (TLC) and H1 is checked on all 36,036 committed records (`hypothesis_gates`: 0 violations)
`detect_a1` sound & complete	Verus spec	Verus	scorer (port)	**proof** against the restatement; restatement = Definition 1 under Lemma 1 (TLAPS step); port gated on every cell (`predicate_matrix_a1`); **port** not yet wired
`detect_a2`	Verus spec (id, signature)	Verus	L_4 dispatcher, scorer	**proof**; Definition 3 (identifiers) coincides under the re-sign-as-replace encoding; **encoding stated, not proved**
`detect_a3`	Verus spec (unsupported read)	Verus	scorer	**proof** for the screen; = Definition 4 under value-unique writes, which hold on every scored trace (`hypothesis_gates`: 600 sessions, 0 collisions)
`detect_a6`	Verus spec	Verus	scorer	**proof**; single-record equivalence to Definition 5
L_1 stores window-free	Verus	Verus (lock predicate)	pilot (bytes); live *verified* arm (bytes via bridge); *validated* arm (application layer)	**bytes** in the pilot and the verified arm; **port** in the validated arm; refinement to the abstract machine **open**
L_2 output commit	Verus	Verus	`l2_exec` (bytes)	**bytes**; sequential object, caller serializes
L_3 sequencing	Verus	Verus	twin (inspection)	**port**
L_4 validation at dispatch	Verus	Verus	`l4_exec` (bytes)	**bytes**
R on the live records	Definition 6	—	`a7_rescore` (Python)	**port**, unverified; independent of the outcome column; extractor recall 4,500/4,500
Basis ledger excludes R	TLA$^+$ G_7; Verus model	TLC 160 runs; Verus verified (7 obligations)	probe 303 ledger arm (Python)	**model** and **proof**; executable ledger is a **port** (harness code)

Section 6.2 measures). The deployed stores do not rest on these: each is an exec-mode store whose only access path is `vstd::rwlock::RwLock`, the reader–writer lock implemented and verified inside the Verus standard library, carrying the store's invariant as the lock predicate (`lib_si_concurrent.rs`, 18; `lib_pess_concurrent.rs`, 31; no axiom, no `external_body`). The invariant entails the absence of any stale-generation window in the store's own record stream, under any interleaving of threads. Theorem L1j (`lib_si_projection.rs`, 3) transports that guarantee to the detector's history for *faithful* emitters—one `OpRecord` per store record, cells a subset of the record's. The default-SI `tick` path is the standing counterexample to faithfulness and is exactly the 5%/8% residual of Table 4. Four earlier refinements (86 obligations, two axioms) relate the abstract machines to the representations the runtime shipped *before* the verified-lock rebuild and bear no deployed claim; refining the two concurrent stores to the abstract machines is the open proof step (a bounded simulation over 506,880 transitions finds no counterexample; supplement § H).

5.3 L_2: output commit

The L_2 model (`lib_l2_safety.rs`, 33) carries per transaction a predecessor set recorded as a causal closure at read time. A commit is valid iff the read set is fresh against the committed store and every predecessor is committed and unaborted; an abort cascades to every transaction whose closure holds the aborter; effects leave by a separate `externalize` step enabled only once the closure has externalized; an externalized transaction cannot abort. Theorem L_2h: no state satisfying the inductive invariant *inv_output_commit* contains a witness of Definition 4 (`lemma_l2_reachable_no_a3`), under arbitrary interleaving of the six steps; the closure invariant makes the one-level cascade provably sufficient for the transitive case. Theorem L_2i: every committed read has a surviving producer of exactly its value, and (temporal strengthening) that producer committed no later than the read. Theorem L_2j (`lib_l2_projection.rs`, 3): a faithful emission of an L_2-supported state contains no residue witness, by a bridge that never refers to a read's producer and so cannot assume what it proves. The exec runtime (`lib_l2_exec.rs`, 73 end to end, 40 net of the re-included model) refines the model per transition with zero added axioms (its `u64` interning is natively injective); every mutating entry point checks its own guard and reports whether it acted, so the guarantee survives proof erasure under plain `cargo`, and the file compiled by the reference runtime is byte-identical.

5.4 L_3: sequencing, and L_4: validation at dispatch

Theorem L_3g (`lib_l3_sequencer.rs`, 5): a commit-order sequencer that externalizes effect k once effects $0..k$ have completed yields an externalization time monotone in issuance index for *every* completion schedule, hence no A_6, without forbidding concurrency; a reversed schedule exhibits A_6 unsequenced and none sequenced. The saga model (`lib_l3_safety.rs`, 6) is the strict-serialization special case and composes with L_2 on disjoint carriers. The exec sequencer (`lib_l3_exec.rs`, 7) is verified; its measurement (Section 6.4) runs a dependency-free twin, a gap the trust base records.

Resolving a planned tool against a registry snapshot does not prevent A_2: a snapshot changes what an operation resolves, not the tool its call reaches (Theorem L_4d, a concrete execution). The L_4 model (`lib_l4_safety.rs`, 9) reads A_2 from a *dispatch record* and validates at dispatch—place the call only if the planned tool is still live with the pinned signature; registry churn interleaves arbitrarily and cannot create A_2 because A_2 is a property of the dispatch. The exec dispatcher (`lib_l4_exec.rs`, 11) refuses a

revoked or re-signed tool and dispatches an untouched one, and Section 6.5 measures those bytes. Validation and dispatch are one atomic step of a registry the runtime owns; a remote tool server (MCP [25]) offers no such atomicity and the check-then-call gap there is narrowed, not closed.

5.5 The basis ledger

The ledger ℓ answers R and composes with any discipline; the live study runs it over L_1 (Table 3). Its model (`lib_l5_ledger.rs`, 7 obligations, 0 errors) carries the store's committed values and, per effect, its basis map, an externalized flag and an obligation flag. Three steps: *register* commits an effect with its basis (released at once, or held under output commit; if released at once on a basis the store already disagrees with, the obligation is set at commit); *write* installs a value and marks, in the same step, every externalized effect whose basis holds that cell with a different value; *externalize* releases a held effect and marks it if its basis is stale by then. The invariant *inv_l5*—every externalized effect with a stale basis carries an obligation—is preserved by each step (the write step is the load-bearing case: the cell that became stale is the cell the step marked, and any other stale cell was stale before), and the ledger theorem: no state satisfying *inv_l5* contains a witness of Definition 6. The model's witness is the state form—an externalized, unowed effect whose basis the store now disagrees with—which contains Definition 6's (a latest post-commit writer that differs implies a current value that differs), so excluding it excludes the definition's witness and, in addition, an effect released on a basis already stale at commit. Non-vacuity in both directions: the unguarded write reaches R in two steps from an effect derived from an unwritten cell, and under the ledger the same write registers the obligation while a write that restores the value leaves no witness. The ledger refuses nothing; refusing to release a stale-basis effect, or holding it for a deadline, is a policy the executable runtime layers on top, and what a registered obligation achieves in the world is measured in Section 6.1. The model is verified (7 obligations) under the pinned build; its obligations are counted as scaffolding (135 with it) because the executable ledger of the live study is harness code, not these bytes. An exec ledger on the L_2/L_4 pattern is not in this version.

5.6 Composition

`lib_consistency_lattice.rs` takes the product of the L_2, L_3 and L_4 states over their disjoint carriers and proves that the conjunction of the three invariants excludes A_1 (image), A_3, A_6 and A_2 jointly, with three non-interference lemmas; 53 obligations, of which 48 are the re-verified component modules. The ledger's carrier (basis, obligation) is disjoint from all three, and adding it to the product is the same construction; it is not in this version.

5.7 Trust base

"Zero `assume`, zero `admit`" is a statement about proof bodies. What is trusted, in full: (i) Verus, its SMT backend and `vstd`, including `group_hash_axioms` for the `HashMap`-backed exec states; (ii) two axioms in the four superseded refinements (string injectivity, null sentinel) and (iii) three `external_body` stubs for a `parking_lot::Mutex`, neither bearing a deployed claim since the verified-lock rebuild; (iv) the hand restatement of the TLA$^+$ predicates in Verus, with the gap of Section 5.1 stated and gated; (v) the hand-written scorer that produced the tables, gated against the verified predicate on every committed cell; (vi) for L_3, the inspection-level correspondence between the exec artifact and the measured twin; (vii) the `pub` fields of the exec runtimes, which a caller can write around the API; (viii) the marshalling that flattens the version map into the verified validation gate, and the interning bijection and abort counter outside the verified store; (ix) the Python and Rust instrumentation that produces every `OpRecord`, including the read sets it declares—a detector is as good as the read set it is handed, and recovering implicit read sets is a separate problem [55]; (x) the provenance correspondence between the two formalizations of A_3, which agree only under value-unique writes (checked on every scored trace); (xi) in the live study's verified arm, the 100-line JSON bridge and the harness's interning of cells and values. No safety-bearing proof invokes a writes-as-function-of-reads assumption, and the development carries no probabilistic obligation.

6 Evaluation

Three questions. **RQ1** For each discipline, what does it prevent, and what harm remains once it has? **RQ2** What does prevention cost, per unit of work the discipline actually delivered? **RQ3** Where do the structural preconditions of the anomalies occur in deployed code? A verified arm's zero is a theorem, so every zero below is reported beside a count that reads no runtime flag.

Provenance. Every table cell maps to a committed trace directory and a scorer in the artifact; `reproduce.sh` rescores all of them offline under an implementation of Definition 1 independent of the verified detector and the Rust port, and fails on any disagreement. The numbers in this section were re-derived that way for this version. Model-driven cells are one run each at the stated n; replicates differ by model sampling, so a regeneration reproduces the protocol and the intervals, not the sessions. Intervals are exact Clopper–Pearson unless stated.

6.1 RQ1 on a live runtime: what L_1 removes and what it leaves

The scenario of Section 1.1 runs on an unmodified LangGraph runtime (`langgraph` 1.2.0) with live models. Two compiled graphs share one `BaseStore`, the framework's cross-thread memory, subclassed only to stamp a logical clock at `get`/`put` under one lock; `BaseStore` offers no version or compare-and-set, so the framework as shipped cannot validate a read set. A flight agent reads the trip date, drafts a reservation over a model call and puts the booking; a date agent, started by the harness 150 ms after the flight agent's read, reads the date, asks its model for a one-week revision and puts it. Three modes: *vanilla*; *validated*, the L_1 discipline at the application layer (re-read before `put`, discard the generation and regenerate on change); and *sequential*, a control in which the date agent runs after the flight agent commits. 100 sessions per cell on three model families. Each session records the A_1 verdict, the regenerations, and the booking's and final trip dates (Table 3).

The window is open and the framework does not close it. On both API models the runtime committed the stale booking in every session in which the revision landed inside the window (60% and 91%), the detector fired with the exact temporal witness of Definition 1, and in 150 of those 152 firings the stale date reached the booking text; nothing in the framework blocked, validated or reported any of them.

TABLE 3
Live cross-agent stale generation on an unmodified LangGraph runtime, n=100 per cell at a 150 ms race offset; the date agent revises in every session. A_1: Definition 1 on the emitted records. R: Definition 6 by argument provenance. *Booking ≠ final*: the harm, read off the store. *Regen.*: discarded generations. Modes: *validated* is L_1 at the application layer; *verified* takes the commit decision from the verified L_1 store through a JSON bridge; *ledger* adds the basis ledger and a rebooking compensator. The first three blocks' vanilla/validated/sequential rows are the committed September run; the verified, ledger and deepseek-chat rows are an October run of all four arms (its own vanilla/validated arms: $A_1 = 94, 44$; residue 12, 51). Every R cell is re-derived from the records (`a7_rescore --check`, 39/39 probe-303 cells agree); under the ledger each superseded basis carries a registered obligation, so R is 0 by Definition 6 and the outcome column is the compensator's.

Model	Mode	A_1 (95% CI)	R	Booking ≠ final	Regen.
gpt-4o-mini	vanilla	60/100 [49.7, 69.7]	40	100/100	0
	validated (L_1)	0/100 [0, 3.6]	34	34/100 [24.8, 44.2]	66
	verified (L_1, verified store)	0/100 [0.0, 3.6]	51	51/100 [40.8, 61.1]	49
	ledger (L_1+ℓ)	0/100 [0.0, 3.6]	0	0/100 [0.0, 3.6]	55+45 (45/45 obl.)
	sequential	0/100 [0, 3.6]	100	100/100	0
claude-haiku-4-5	vanilla	91/100 [83.6, 95.8]	9	100/100	0
	validated (L_1)	0/100 [0, 3.6]	7	7/100 [2.9, 13.9]	93
	verified (L_1, verified store)	0/100 [0.0, 3.6]	2	2/100 [0.2, 7.0]	98
	ledger (L_1+ℓ)	0/100 [0.0, 3.6]	0	0/100 [0.0, 3.6]	95+5 (5/5 obl.)
	sequential	0/100 [0, 3.6]	100	100/100	0
Llama-3.2 (local)	vanilla	1/100 [0.0, 5.4]	99	100/100	0
	validated (L_1)	0/100 [0, 3.6]	100	100/100	0
	sequential	0/100 [0, 3.6]	100	100/100	0
deepseek-chat	vanilla	45/100 [35.0, 55.3]	55	100/100 [96.4, 100.0]	0
	validated (L_1)	0/100 [0.0, 3.6]	63	63/100 [52.8, 72.4]	37
	verified (L_1, verified store)	0/100 [0.0, 3.6]	52	52/100 [41.8, 62.1]	48
	ledger (L_1+ℓ)	0/100 [0.0, 3.6]	0	0/100 [0.0, 3.6]	47+53 (53/53 obl.)
Pinned interval, read to commit, s (mean; max): gpt-4o-mini 1.07; 3.79 haiku 1.21; 1.96 Llama 3.77; 9.06					

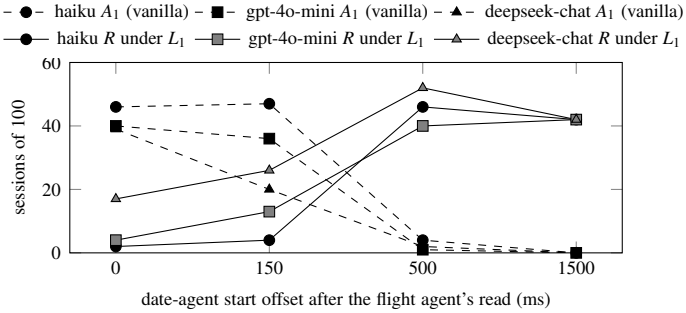


Fig. 3. Probe 303, offset sweep: the live A_1 rate without a discipline (dashed) and the residue R that read-set validation leaves (solid) as the date agent's start is delayed; revision probability 0.5, $n = 100$ per point. As the window closes the same revisions land after the commit: A_1 falls to 0 while R rises to the revision rate.

What the rates are. Race outcomes: the revision changed the value in 300/300 sessions, and whether it landed inside the window is the reviser's latency relative to the writer's plus the fixed offset. The open-weights family makes this explicit: behind a request-serializing local server the revision landed after the commit in 99/100 sessions, the one firing being the session with the longest pinned interval (9.06 s)—a property of the serving configuration, not of the model.

What L_1 removes, and what it does not. Read-set validation removed every firing (0/300) at one discarded generation per realized conflict (66 and 93 per 100 sessions, never two in one session, none where the race never realized). It did not make the booking agree with the final trip date: in 34 and 7 of 100 sessions the revision landed after the commit: the committed record is current at its commit point exactly as Theorem 1 says, and the booking is stale with respect to a state that did not yet exist. That residue is a post-commit supersession—the blind overwrite of Section 3.1, serializable in commit order, outside every isolation discipline—and only compensation, or holding the effect until its basis is final, reaches it. The *sequential* control, clean under every commit-time predicate, delivers the wrong booking in 100/100 sessions for the same reason; reporting the predicate alone would have credited L_1 with a correctness it does not provide.

***R* decides the residue on the records.** Definition 6 names what is left, and the R column of Table 3 is decided on the emitted records alone: the basis of each booking is the date value in its arguments, and R fires in exactly the sessions whose booking disagrees with the final date and A_1 does not ($A_1 + R = 100$ in every unguarded cell; under L_1 the R column equals the residue; `a7_rescore.py --check`). The identity is between two independent computations (Section 3.6), and it holds here because the booking tool takes the date as a structured argument; a workload whose effects paraphrase their inputs would show a recall below 4,500/4,500 and a smaller R column, never a larger one. Two further arms run the same harness on three API families (haiku, gpt-4o-mini and deepseek-chat, the last through its OpenAI-compatible endpoint). In the *verified* arm the L_1 decision is taken by the verified store through the bridge of Section 5.7: A_1 fires in 0/300 sessions, and the residue is 2, 51 and 52 of 100—the same kind of number the application-layer arm leaves in the same run (12, 51, 63), the haiku pair differing by run-to-run sampling of which revisions land after the commit, and the two runs of the validated arm on gpt-4o-mini (34 in September, 51 in October) showing the same spread. In the *ledger* arm a superseding put registers the obligation and a compensator issues a replacement booking: the outcome column is 0/100 on all three families (5, 45 and 53 obligations registered, each discharged by one compensation generation), at the price of the discarded generations of L_1 plus one generation per obligation.

The offset sweep. Figure 3 varies the race instead of fixing it: the

TABLE 4
AutoGen pilot, A_1 firing rate by runtime and workload (run `20260913T0150Z`, n=100 per cell). *Complete & clean*: sessions with no A_1 witness in which every role that commits in the unguarded cell also commits here.

Runtime	edit-review	plan-execute	triage
Vanilla	100% [96.4, 100]	0% [0, 3.6]	48% [37.9, 58.2]
Pessimistic locking	0% [0, 3.6]	0% [0, 3.6]	0% [0, 3.6]
Snapshot isolation (default-SI)	5% [1.6, 11.3]	0% [0, 3.6]	8% [3.5, 15.2]
Complete & clean, vanilla / pess. / SI	0 / 1 / 0	99 / 100 / 99	52 / 3 / 45

date agent starts 0, 150, 500 or 1,500 ms after the flight agent's read and revises in about half the sessions ($p = 0.5$), $n = 100$ per point on the three API families. Without a discipline A_1 fires in 39–46 of 100 sessions at 0 ms and in 0 at 1,500 ms, where every revision lands after the commit, while the booking is wrong in every revised session at every offset (44–51). Under L_1 the curve inverts: the residue is 2–17 at 0 ms and 42–52 at 500–1,500 ms, the discarded generations falling from 34–49 to 0. Predicate and residue are the two halves of one quantity, the revision rate, split by where the revision lands relative to the commit; which half a deployment sees is a property of its latencies, and the ledger is the only arm whose outcome does not depend on them. What Proposition 1 says must remain after release does remain under every commit-time discipline, and the ledger is the one arm that answers it.

Superstep. A second graph places a writer and a summarizer of one reducer-declared channel in one Pregel superstep [26]; the barrier hands both the snapshot at superstep entry, so in 300/300 sessions the summary names the destination the writer had changed. This is the snapshot-insufficiency shape of Section 3.5 observed live (a reducer buys lost-update freedom, not L_1), reported as documented BSP semantics, not a rate: the census finds it in no public graph, and under a completion clock Definition 1 fires exactly when the writer's body returned first (42, 72, 26 of 100; `check_clocks.sh`).

6.2 RQ1 across three disciplines: the AutoGen pilot

An instrumentation layer exposes a two-phase `begin`/`commit` API to LLM-driven agents under AutoGen [19]; the Rust runtimes of Section 5.2 sit behind it. Three workloads span the space from concurrent to sequential: *edit-review* (two agents both read `doc` before either writes—overlap forced), *plan-execute* (strictly sequential—overlap forbidden) and *triage* (a three-agent pipeline whose overlap is left to the models). gpt-4o, 100 sessions per cell, three runtimes (Table 4); the vanilla column was replicated three times (triage 42%, 48% and 52%; pooled 142/300). A Claude Sonnet 4.5 rerun of the vanilla column with operation records reproduces the two determinate cells (edit-review 100/100, plan-execute 0/100); its triage cell is void, the reporter having written a ticket without a description in 100/100 sessions (supplement § M).

Both endpoints are engineered, so the table is a sensitivity check of the detector and the runtimes on model-driven traces, not a prevalence. Every edit-review witness is a cold-start read (the reviewer read the document before the editor's first write landed), unlike the live study's supersessions. Default-SI's residual is its no-write bypass: a read-only commit is never validated, which is the classical write-only SI behavior and the unfaithful emitter of Theorem L1j; SSI mode closes it. Pessimistic locking reaches 0% on edit-review by refusing the reviewer's operation in 99/100 sessions—the anomaly is absent because the work is—and on triage by dropping one of three agents in 97/100. The *complete & clean* row is the number to read a prevention claim against, and on triage it favors the unguarded baseline (52) over default-SI (45) and pessimistic locking (3), because default-SI drops the engineer in 47% of sessions and pessimistic locking in 97%. Whether a dropped contribution or a stale one is the worse outcome is a property of the task, not of the discipline, and is why Section 6.6 prices prevention per sound committed operation.

6.3 RQ1 for L_2: output commit on synthetic cascades and live verdicts

Three arms run the same 1,000 seeded schedules at nominal chain depths 2, 3 and 5: a root commits, a chain of dependents reads the previous cell and commits, one committed transaction is put under review and retracted or approved, and in a quarter of scenarios the review arrives after that transaction has released. The *verified* arm is `l2_exec.rs`; the *unguarded* arm validates reads, releases at commit and flags only the aborted transaction; the *cascade* arm releases at commit and cascades the abort. Each trace is scored under Definition 4, under the surviving-dependent predicate, and by *exposed effects*—effects out at the end although the review asked to withdraw their basis—which reads no flag (full table: supplement § P). The result is one structural fact. Definition 4 fires in 519, 624 and 563 of 1,000 schedules per depth under both baselines and in none under the verified arm; the cascade arm satisfies its own surviving-dependent predicate in every scenario while exposing exactly as many effects as no discipline at all (1,538, 2,059 and 2,010), because it flagged 767, 1,288 and 1,239 dependents *after* their effects were out—a cascade without output commit moves flags, not effects. The verified arm is not zero on exposed effects: 402, 547 and 549 effects in 199 scenarios per depth, exactly the scenarios in which the review arrived after the release and the retraction was refused. The figures decompose by arithmetic ($1{,}538 = 402 + 572 + 564$ at depth 2): the first term no runtime can remove; the second, one reviewed operation per early review, any runtime that declines to release an operation under review avoids; the third (37–46% of the unguarded exposure) is output commit's own contribution, dependents whose release it refused and the cascade then cancelled. Its cost is delay, not liveness (748–1,272 operations had a release refused at least once; every committed, unaborted operation was eventually released). The quarter of late reviews is a harness parameter, so the overall exposure reduction (73–74%) is that parameter seen from the other side, not a finding.

Live verdicts. A plan–execute–revise workload driven by real models (a planner commits a plan; an executor reads it and commits, acquiring the planner as predecessor; a supervisor decides whether to retract) was recorded for 200 sessions on each of three families and replayed through the three arms. Retraction rates varied sharply—1/200 (claude-haiku-4-5), 3/200 (gpt-4o-mini), 62/200 (Llama-3.2)—so we make the pooled statement only: in every one of the 66 retracted sessions the unguarded baseline and the cascade design left a Definition 4 witness (the latter while satisfying the surviving-dependent predicate), and the verified runtime released no executor effect on a retracted plan, at a pooled release rate of 89.0%. A further 1,000 haiku sessions produced no retraction at all, so on that family the discipline's only observable cost is the hold: each kept effect waited for the supervisor's verdict, a median 687 ms (p90 789 ms). The zero is structural; the replay

TABLE 5
Model-emitted batches through LangGraph's released `ToolNode` (`langgraph` 1.2.5), 30 turns per backbone, natural prompt. Tool latency modeled. *Transcript*: turns in which the returned `ToolMessage` list preserved issuance order although effects committed out of order.

Backbone	batched	full pipeline (5)	A_6, default	A_6, L_3
gpt-4o-mini	30/30	18 (18 reordered)	22/30	0/30
Llama-3.2	3/30	0	2/3	0/3
claude-haiku-4-5	0/30	0	—	—

Transcript preserved issuance order in 33/33 batched turns; 24 of them had committed out of order.

shows the discipline executing under supervisor behavior that ranges from $0/1{,}000$ to $62/200$ retractions, and the A_3 hazard itself is a property of the backbone's retraction rate.

6.4 RQ1 for L_3: a released tool executor reorders

LangGraph's `ToolNode` dispatches a turn's tool calls concurrently (`asyncio.gather`), collects results in input order and does not serialize side effects; its tracker records a user needing sequential execution for stateful tools and unable to obtain it (langgraphjs #861). We gave three backbones a natural prompt (no instruction to parallelize) over five order-dependent infrastructure tools whose docstrings state their dependencies, took the model-emitted call list as *io*, and dispatched it through the released `ToolNode` and through the L_3 sequencer. In the table's cells two things are real—whether a backbone batches an order-dependent pipeline into one turn, and the executor—and per-tool latency is modeled, drawn uniformly from 10–200 ms (a uniform-latency control produces no reorder; the committed order is the latency argsort). The replay against real effects below removes the model.

The finding that survives the modeling is structural (Table 5): one backbone batches the full dependent pipeline on $18/30$ turns and reorders every one of them, another never batches, and the transcript a developer inspects shows the correct order in all 33 batched turns while the external state does not. Calls emitted in one turn are, by the provider's contract, calls the model declared parallelizable, so the defect is shared between a backbone that misjudged independence and an executor that cannot notice; the L_3 sequencer makes effect order independent of which backbone is in use, which is the property the cross-model spread shows cannot otherwise be assumed. Replaying the 33 recorded batches five times each through `ToolNode` against real effects—a WAL-backed SQLite database and a local HTTP service, with whatever latency those have—rather than a modeled sleep, baseline A_6 fires in $126/165$ replays (SQLite) and $112/165$ (HTTP), and L_3 in none: the reorder is a property of concurrent dispatch under any real latency, not of the distribution we drew.

6.5 RQ1 for L_4: validation at dispatch

One seeded schedule per run pins a tool and then revokes or re-signs random tools; three dispatchers act on the same registry. Over 1,000 runs at registry widths 2, 4 and 16 the churn touched the planned tool in 630, 505 and 387 runs, and in exactly those runs both live resolution and snapshot resolution called a phantom—Theorem L_4d measured rather than argued—while the verified dispatcher (`l4_exec.rs`, the verified bytes) called none and refused exactly those runs. The informative figures are the hazard rate and the refusal cost; the zero is Theorem L_4b executing.

TABLE 6
Token cost of prevention under three denominators, gpt-4o, run `20260913T0150Z`. Per sound committed operation is the price we recommend reading. Underlying counts (commits / sound): edit-review vanilla $212/112$, pessimistic $101/101$, SI $110/105$; triage vanilla $335/283$, pessimistic $205/205$, SI $268/256$; plan-execute $\approx 200/200$ for all three.

Workload	Discipline	per session	per commit	per sound commit
edit-review	pessimistic	+3.8%	+117.9%	+15.1%
edit-review	SI	+0.6%	+94.0%	+7.4%
plan-execute	pessimistic	+1.7%	+1.2%	+1.2%
plan-execute	SI	+0.4%	+0.4%	+0.4%
triage	pessimistic	+42.7%	+133.2%	+97.0%
triage	SI	+0.5%	+25.6%	+11.1%

6.6 RQ2: what prevention costs

The pilot driver captures per-call usage and wall-clock for every session of run `20260913T0150Z` (gpt-4o, 900 sessions) and of its Claude Sonnet 4.5 replication; pricing is list price at the time of the run. Per session, snapshot isolation's token cost is not separable from the unguarded runtime's on either provider (overlapping intervals in every cell), and pessimistic locking's overhead is provider-dependent (at most $1.6\times$ on gpt-4o and $2.3\times$ on Claude, both on triage, where it retries). A per-session denominator, however, credits a discipline for work it declined to do. Table 6 re-prices the same run per committed operation and per *sound* committed operation—one that is not the stale reader of any Definition 1 witness—so that no runtime earns credit for output an anomaly has invalidated.

Between-session parity does not survive the change of denominator: snapshot isolation costs +7.4%, +0.4% and +11.1% per sound committed operation, and pessimistic locking's triage penalty is +97% rather than +43%. Overhead tracks realized contention: edit-review, which aborts hardest, pays most; plan-execute, which never aborts, pays nothing under any denominator. A separate 155-session paired sweep on gpt-4o-mini (fan-in 2–16 at one cell; 2–32 cells at fan-in 8; realized abort rates 0.09–0.93) resolves the mechanism: snapshot isolation carries no fixed overhead (intercept +0.05%, CI $[-1.5, +1.7]$) and pays $1.074\times$ $[1.052, 1.096]$ a first-attempt generation per abort, because the retry carries the conflict into its prompt; the per-abort price does not steepen with contention (supplement § M).

Wall-clock, with its confound. The nine cells of a run were executed one after another, so a difference between cells carries whatever the provider's latency did in between. Plan-execute, on which no runtime aborts, is the control, and its "overheads" are −4.5% and −4.9% on gpt-4o (Holm-corrected permutation $p =$ 0.014, 0.002) and +13.5% for SI on Claude ($p = 0.001$): that is drift, and we report its largest magnitude as the run's null band. Outside it, on mean and median alike: pessimistic locking at +16.7% (edit-review) and +34.9% (triage) on gpt-4o and +109.5% (triage) on Claude, where it makes 7.75 model calls per session against 4.00. Snapshot isolation is inside the band in every cell on both families, which is not the same as zero: the design cannot resolve an overhead below about 5% on gpt-4o or 14% on Claude, and the per-sound-commit wall-clock overheads on gpt-4o (+4.9% and +11.0%) sit exactly there. Only a rerun with the three runtimes interleaved per session removes the confound; probe 302 is that design, and its paired per-session overheads (mean, bootstrap 95% CI; "–" until run) are, on gpt-4o, triage pessimistic – and SI –, with the plan-execute control at – and – (supplement § O for every cell).

TABLE 7
Census of public LangGraph repositories (frame of 19 September 2026, seed fixed). Repositories unless marked; Wilson 95% intervals. *Confirmed*: a co-scheduled reader and writer of a last-value channel. *Upper*: also pairs with an unresolved side.

Star band	w/ graph	sites	parallel	confirmed	upper
≥ 200	39	248	7	0	4
10–199	28	111	1	0	1
0–9	31	48	3	0	3
All	98	407	11	0	8
share			11.2%	0%	8.2%
95% CI			[6.4, 19.0]	[0, 3.8]	[4.2, 15.3]

6.7 Operational materiality of a firing

A value mismatch may be lexical. Holding an agent's context fixed and varying only the shared value it read, the decision changed in $150/200$ flagged mismatches of a controlled triage task (75.0% [68.4, 80.8]); divergence tracks whether the value pair straddles a decision boundary, so the rate is a property of the pairs sampled as much as of the agents, offered as the paper's only value-staleness estimate, not a constant. On the pilot's Definition 1 firings (all cold-start reads by the reviewer) materiality is $43/43$ under an LLM judge, blind-coded by hand on 15; a human-judge agreement of $59/60$ on a broader sample says the judge reads decision-level divergence, not that those 43 are validated.

6.8 RQ3: where the preconditions occur

Every occurrence figure above is a race we opened or a rate on our own workloads. To put a denominator under the structural precondition of A_1 in the framework we studied—a reader and a writer of one last-value channel released into the same superstep—we analyzed public code. *Frame*: GitHub repositories matching `langgraph` in name, description or readme (Python; forks and the framework's own organization excluded), in three star bands; 60 drawn per band under a fixed seed from the API's 500 most recently updated. *Instrument*: a static front end that recovers, from the syntax tree alone, each `StateGraph` site, the node pairs one fork releases into a superstep, and each node's read and written keys; whatever it cannot resolve is *unknown*, never guessed (eleven fixtures pin its rules; a differential oracle agrees with execution on the eight runnable ones).

All 180 sampled repositories were read; 98 contain a visible graph, holding 407 sites, of which 392 (96%) are sequential. Fifteen sites in 11 repositories contain a co-scheduled pair; none is a confirmed reader/writer pair on a last-value channel (repository-level upper limit 3.8%); two are such pairs on a reducer channel, the ordinary parallel-agents-on-`messages` idiom; eight have an unresolved side, bounding the susceptible share at 8.2% of repositories. Counting every unresolved site (82 with routing the front end could not resolve, 7 with a `Send` fan-out) gives a wide bound of 51%; restricting to the 251 source-path sites changes nothing. An enriched population—the 34 third-party repositories whose issues quote LangGraph's co-superstep writer/writer error, projects that demonstrably ran parallel writers—has parallel graphs in $12/26$ but again no confirmed site in 391. The bracket is *none confirmed, at most about half*; the narrow reading is a negative result for the motivation of Section 6.1, recorded as one. The census measures how code is written, not how often a graph runs or whether a co-scheduled read goes stale.

Mechanisms, not rates. Of five deployed designs we probed, four close the window and one leaves it open: Letta re-renders the shared block into every generation step; LangGraph's `interrupt` re-executes a node from its start on resume; CrewAI rejects a concurrent read of a concurrently produced value at construction; AutoGen's actors own their state. LangGraph's superstep channels and cross-thread store pin a read across a node body that may contain a model call, which is where Section 6.1 measures it. Five systems is not a sample; each entry names a mechanism a designer can choose. Two framework defects outside the catalog were found on the way: the widely cited deer-flow bug (#3123) is a sequential blind overwrite under a last-value channel, not a concurrency anomaly, and `interrupt`'s re-execution emits an effect issued before the gate twice (supplement § Q).

6.9 Threats to validity

Construct. A_1 tracks the window, not the harm; Table 3 reports both and R names the gap. The basis is argument provenance, verbatim and not semantic. Whether a booking that disagrees with a later revision is harm is the application's call; we count it because the user asked for one trip. A_6 fires on commuting effects and is silent on dropped ones. *Internal.* The live rates are latency races, $n = 100$ single runs per cell, at one offset in Table 3 and four in Figure 3; the wall-clock cells carry a run-order confound we bound rather than remove (probe 302 removes it; not run); the L2 live replay rests on 66 retractions, 62 from one family (1,000 further haiku sessions added none). *External.* Three API families and one local; one framework driven live; toy workloads of two to three agents. *Reproducibility.* Every cell is committed with its traces and rescored by an independent implementation of Definition 1; run `20260913T0424Z` carries a manifest captured at run time, the two earlier runs carry reconstructed manifests that say so.

7 Related work

7.1 Classical consistency and isolation

The hardware tradition [5]–[8] characterizes visibility under bounded latency; ours is unbounded. The database isolation hierarchy [9]–[11] is the direct ancestor: we adopt its anomaly-based method over a different operational model. The line through Fekete et al. [23] and Cahill et al. [17], productized in PostgreSQL [28], is the reference for our SSI runtime; the difference is that the work discarded on abort is an inference. Weikum and Vossen's commit-order-preserving serializability [3] and Kung and Robinson's backward validation [4] are what Theorem 1 mechanizes at the level of values read; Papadimitriou's view equivalence [29] is where the value-level and conflict-level notions meet. Bailis et al.'s highly available transactions [102] classify isolation levels by the anomalies they admit under availability constraints, the taxonomy our lattice borrows its method from; Cerone, Bernardi and Gotsman [31] give the axiomatic framework our history predicates specialize; *robustness*—whether every execution of an application under a weak model is serializable [32], [33]—is what the census of Section 6.8 asks statically; our detectors are polynomial [34] only because each predicate is existential over a constant number of records. Crooks et al. [30] give the client-centric formulation we do not adopt. Compensation descends from sagas [22], ConTracts [70] and Gray and Reuter [85]; recoverability

and strictness are Hadzilacos [61] and Bernstein et al. [21]. Irrevocable transactions [37] and transactional boosting [86] met the same irreversibility in transactional memory; output commit is the dual choice, delay the effect and keep the operation abortable. Commutativity-based concurrency control [103] orders only what does not commute, and an A_6 discipline over commutativity classes rather than issuance order would pay the sequencing cost only where it buys something; transaction chopping [104] says when a long operation may be split, the question an agent's multi-step plan raises. The basis of Definition 6 is why-provenance [105], [106] restricted to copying; that literature's treatment of derivation through computation is what a semantic basis would need. Exactly-once stream processing [108], [109] holds side effects at a transactional sink until the input is durable, the industrial precedent for output commit. The pessimistic/optimistic choice our cost analysis turns on is the contention-management problem of STM [35], [36], and the cost-aware selection of a level is consistency rationing [87] and, in databases today, TxnSails [84].

7.2 Concurrency control and shared memory for LLM agents

A 2025–2026 line builds mechanisms for this regime, mostly without verifying them, and the present paper is one point in it rather than its origin. Agentic transactions [111] recast ACID as four semantic guarantees for a single long-horizon agent (explore–execute–validate cycles, dependency-aware isolation) and measure task success, with no history predicate, model or proof; our catalog is the anomaly-level account such guarantees would be checked against, and supplement § D places it on the lattice with the systems below. CoAgent [53] states the regime in our three terms and proposes an advisory protocol in which a notified agent repairs its own plan; its correctness argument is conditional on a model judgment measured at a 5% failure rate, and its lock/OCC cost inversion independently reproduces Section 6.6. STORM [54] is read validation (L_1) deployed per file; ATCC [56], OptiMA [57], token coherence [58] and GoEX [59] impose a remedy from the runtime. MemTX [88] is the closest design to our L_2: staged writes under snapshot isolation, irreversible tool calls gated on current belief and cascading repair on retraction, with two invariants checked by bounded enumeration (5.5M states) and a five-backbone evaluation; it does not state the predicates it excludes, verify the runtime, or price prevention per unit of sound work. Yang et al. [89] argue that coordination failures attributed to communication are concurrency failures; governed shared memory [90] and transactive memory [91] address provenance, supersession and scoping without treating the generation window as a transaction. SagaLLM [1], Atomix [2] and CodeCRDT [27] are placed on the lattice informally in supplement § D; none reaches L_4. S-Bus [55] (the author's prior work) recovers implicit read sets for one contract and is the natural supplier of trust-base item (ix). Agent-memory systems [69] and the frameworks [19], [112] supply substrate, not contract.

7.3 Commit-time guards, retraction, and what comes after commit

The 2026 work on agent effects acts at the commit boundary: CommitGuard [97] licenses a durable effect only by a witness still fresh at commit; Zheng et al. [98] show that freshness guards of our L_1 kind block 92–95% of benign infrastructure races and argue for semantic commit predicates; EffectMatch [99] validates persistent outcomes against what was approved; versioned execution [100] revokes an obsolete execution's authority to publish. MemTX [88] alone looks past commit: on an explicit *retraction* it walks a derivation DAG to executed tool actions and records each as compensated or leaked. None treats the ordinary case Table 3 measures—no retraction, no race, a legitimate later write—and none states what guarantee is achievable there; Proposition 1 says it is an obligation under an open-world store, and the ledger registers it atomically. Argument-level provenance is used by PACT [101] to decide whether a tool argument's origin is trusted; we use the same granularity for a different question, which value an effect depends on, and the precision objection of Zheng et al. is answered at the same place: a basis predicate ignores writes to cells the effect never depended on.

7.4 Asynchronous tool calling and effect order

Parallel function calling is pursued deliberately: LLMCompiler [48] plans a dependency graph and dispatches independent calls concurrently, and asynchronous function calling [92], [93] lets a model issue calls and continue generating while they complete, with ordering of effects left to the runtime. A_6 is a wrong independence judgment seen at the effect level (Section 6.4: made by the backbone, unnoticed by the executor). The run-to-run spread the evaluation reports is the documented nondeterminism of sampled inference [110]. Sandboxes and policies for tool effects [50], [83], [94]–[96] gate *what* an agent may do, orthogonally to the order and basis of effects; τ-bench [49] scores end state for a single agent and isolates neither.

7.5 Durable execution, exactly-once and repair

Durable-execution engines [51], [52] and exactly-once workflow systems [65]–[68] govern whether an effect is *repeated* after a crash; output commit governs whether it may *leave* while its basis can be withdrawn, and the two compose. Rollback recovery [80], [81] discharges duplicates by identity because replay is deterministic; agent runtimes break that premise [82], and the `interrupt` hazard of Section 6.8 is an instance. Retro, Warp and Aire [62]–[64] repair effects that have already left by compensating requests; a refused retraction in our L_2 is such an effect, and the recorded closure is the dependency information they consume.

7.6 Verified systems and black-box checkers

IronFleet [38], Verdi [39] and Chapar [40] verify running protocols and stores; Perennial, GoJournal and Argosy [71]–[73] verify concurrent, crash-safe storage by layered refinement, the template for the open step of Section 5.2. Ours are far smaller, and what they add is a verified detector and verified bytes executed in a measurement. Jepsen [41] and the black-box checkers [42]–[47] decide serializability or snapshot isolation over database histories with unverified analyzers under a uniqueness assumption we share (Section 3.3); ours decide a far simpler property and are verified against it, in the tradition of runtime verification [74]–[76].

7.7 Agent failure taxonomies and verification

MAST [16] catalogs cognitive and coordination failures; the concurrency class is absent because current benchmarks do not stress shared state under contention, and its traces cannot show A_1 either way (supplement § C). Agentproof [60] analyzes agent graphs statically with witness generation; the census is that programme specialized to one predicate (per-node write sets are not statically derivable in LangGraph). VeriMAP [77], AgentVerify [78] and MCMAS [79] verify task-level and strategic properties, a complementary layer.

8 Limitations and future work

The predicates are coarser than the phenomena. A_1 over-reports in one shape (a value overwritten and restored inside the window; 0 of 710 witness pairs on the live records) and, as Table 3 shows, under-reaches the harm by the post-commit supersession that R names. A_6 fires on commuting effects and is blind to dropped and duplicated ones. The detectors are sound and complete against their Verus specifications; Table 2 says for each where the specification and the catalog definition coincide and under what hypothesis.

R is not an anomaly, and its basis is syntactic. R fires in $100/100$ sequential sessions because the workload revises the date in every session; what Table 3 establishes is that the harm an isolation discipline cannot reach is exactly this predicate, not that concurrency causes it, which is why it stands outside the catalog. Its basis is argument provenance: a value that appears verbatim in an effect's arguments. A booking derived from the date by computation ("the day after") has an empty basis and is invisible, so the extractor under-reaches and the R column is a lower bound; on this workload its recall is 4,500/4,500 because the booking tool takes the date as a structured argument, and a workload with free-text effects needs a declared or taint-tracked basis. Whether a registered obligation is discharged, and at what cost, is a property of the compensator; the ledger's guarantee ends at registration, and the ledger arm of Table 3 measures one compensator on one workload.

A refused retraction is not a prevented anomaly. Output commit makes Definition 4 unreachable partly by holding effects and partly by refusing to retract an operation whose effects are out; the second leaves the world as it was and is counted (Section 6.3). The exposure–latency curve between the harness's one point and the live driver's extreme is unmeasured.

Verification is relative to the trace, and the triangle does not yet close on one system. Trust-base item (ix) is the largest: a stale read the instrumentation did not declare is invisible, and recovering implicit read sets is a separate problem [55]. The scorer is a hand-written port gated against the verified predicate, not the predicate itself; the refinement from the two concurrent stores to their abstract machines is open; L_3 is measured on a twin; the L_2–L_4 exec artifacts are sequential objects a multi-threaded caller must serialize. The verified arm puts verification and the flagship measurement on one system for L_1 only; the ledger arm's executable is harness code. Liveness is verified nowhere.

At-most-once externalization is assumed, and a deployed path breaks it. LangGraph's `interrupt` resumes by re-executing the node, which closes the stale-generation window and emits an effect issued before the gate twice, with different arguments when the shared value was revised during the gate; the catalog has no predicate for duplication and we record it as a limitation of the model.

Prevalence is not established, and the census points the other way. The census bounds a structural precondition in public code (none confirmed; at most about half counting every unresolved site) and says nothing about private deployments, run frequency or the cross-thread store variant; its reading is that public LangGraph code rarely contends. Whether these anomalies occur in production at a rate that justifies the costs of Section 6.6 is open; the live rates are latency races from single runs of $n = 100$ sessions per point.

Not yet run, each a step of the artifact's runner: a multi-run repetition of the live cells; the interleaved cost rerun (probe 302); the Claude pilot's guarded cells with operation records; and wiring the verified detector into the scorer. Longer tasks, more agents and an instrumented deployment remain the empirical follow-up; the store refinements, the formal one.

9 Conclusion

Four classical consistency conditions port to the regime in which an agent's step spans an inference call, and the development is mechanized at stated assurance levels: TLA$^+$ predicates with TLC witnesses, a realizability matrix over every point of their exclusion lattice, a TLAPS theorem placing stale-generation freedom as a conditional commit-order read consistency, Verus detectors sound and complete against their specifications with each specification's relation to its definition named, and four disciplines verified in Verus, two of them executed as the bytes that were verified. What the four leave is not an anomaly: an effect out on a basis later writes revise, which under an open-world store can only be owed, and for which a verified basis ledger registers the obligation atomically at every lattice point and, with a compensator, removes the residue on three API families. The evaluation's contribution is not the verified arms' zeros, which are theorems, but what sits beside them: the harm each discipline leaves (R in 7–51% of live sessions under L_1 at 150 ms and up to 52% as the race offset grows; 402–549 exposed effects per 1,000 cascades under L_2), its cost per unit of sound work ($+0.4$–11% for snapshot isolation, up to $+97\%$ for locking), and where the preconditions occur in deployed code (nowhere confirmed in 407 public LangGraph sites for A_1; $30/30$ turns of one backbone's tool batching for A_6). A discipline should be judged by all three; the artifact re-derives each from committed data.

References

[1] E. Y. Chang and L. Geng, "SagaLLM: Context management, validation, and transaction guarantees for multi-agent LLM planning," *Proceedings of the VLDB Endowment*, 2025, arXiv:2503.11951.

[2] B. Mohammadi, N. Potamitis, L. Klein, A. Arora, and L. Bindschaedler, "Atomix: Timely, transactional tool use for reliable agentic workflows," *arXiv preprint arXiv:2602.14849*, 2026, preprint; not yet peer-reviewed at time of writing.

[3] G. Weikum and G. Vossen, *Transactional Information Systems: Theory, Algorithms, and the Practice of Concurrency Control and Recovery*. Morgan Kaufmann, 2001.

[4] H. T. Kung and J. T. Robinson, "On optimistic methods for concurrency control," *ACM Transactions on Database Systems*, vol. 6, no. 2, pp. 213–226, 1981.

[5] L. Lamport, "How to make a multiprocessor computer that correctly executes multiprocess programs," *IEEE Transactions on Computers*, vol. C-28, no. 9, pp. 690–691, 1979.

[6] K. Gharachorloo, D. Lenoski, J. Laudon, P. Gibbons, A. Gupta, and J. Hennessy, "Memory consistency and event ordering in scalable shared-memory multiprocessors," in *ISCA*, 1990, pp. 15–26.

[7] C. Pulte, S. Flur, W. Deacon, J. French, S. Sarkar, and P. Sewell, "Simplifying arm concurrency: Multicopy-atomic axiomatic and operational models for ARMv8," in *POPL*, vol. 2, 2018, pp. 19:1–19:29.

[8] M. P. Herlihy and J. M. Wing, "Linearizability: A correctness condition for concurrent objects," *ACM TOPLAS*, vol. 12, no. 3, pp. 463–492, 1990.

[9] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil, "A critique of ANSI SQL isolation levels," in *ACM SIGMOD*, 1995, pp. 1–10.

[10] A. Adya, "Weak consistency: A generalized theory and optimistic implementations for distributed transactions," Ph.D. dissertation, MIT, 1999.

[11] A. Adya, B. Liskov, and P. O'Neil, "Generalized isolation level definitions," in *Proceedings of the 16th International Conference on Data Engineering (ICDE)*, 2000, pp. 67–78.

[12] W. Lloyd, M. J. Freedman, M. Kaminsky, and D. G. Andersen, "Don't settle for eventual: Scalable causal consistency for wide-area storage with COPS," in *SOSP*, 2011, pp. 401–416.

[13] S. Burckhardt, *Principles of Eventual Consistency*. now publishers, 2014.

[14] D. B. Terry, A. J. Demers, K. Petersen, M. J. Spreitzer, M. M. Theimer, and B. B. Welch, "Session guarantees for weakly consistent replicated data," in *PDIS*, 1994, pp. 140–149.

[15] P. Viotti and M. Vukolić, "Consistency in non-transactional distributed storage systems," *ACM Computing Surveys*, vol. 49, no. 1, pp. 19:1–19:34, 2016.

[16] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica, "Why do multi-agent LLM systems fail?" in *NeurIPS Datasets and Benchmarks Track*, 2025, spotlight; arXiv:2503.13657.

[17] M. J. Cahill, U. Röhm, and A. D. Fekete, "Serializable isolation for snapshot databases," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2008, pp. 729–738.

[18] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[19] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," in *arXiv preprint arXiv:2308.08155*, 2023.

[20] M. S. Miller, "Robust composition: Towards a unified approach to access control and concurrency control," Ph.D. dissertation, Johns Hopkins University, 2006.

[21] P. A. Bernstein, V. Hadzilacos, and N. Goodman, *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987.

[22] H. Garcia-Molina and K. Salem, "Sagas," in *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, 1987, pp. 249–259.

[23] A. Fekete, D. Liarokapis, E. O'Neil, P. O'Neil, and D. Shasha, "Making snapshot isolation serializable," *ACM Transactions on Database Systems (TODS)*, vol. 30, no. 2, pp. 492–528, 2005.

[24] A. Lattuada, T. Hance, C. Cho, M. Brun, I. Subasinghe, Y. Zhou, J. Howell, B. Parno, and C. Hawblitzel, "Verus: Verifying Rust programs using linear ghost types," in *Proceedings of the ACM on Programming Languages (OOPSLA)*, vol. 7, 2023, pp. 286–315.

[25] Anthropic, "Model context protocol," https://modelcontextprotocol.io, 2024.

[26] G. Malewicz, M. H. Austern, A. J. C. Bik, J. C. Dehnert, I. Horn, N. Leiser, and G. Czajkowski, "Pregel: A system for large-scale graph processing," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2010, pp. 135–146.

[27] S. Pugachev, "CodeCRDT: Observation-driven coordination for multi-agent LLM code generation," *arXiv preprint arXiv:2510.18893*, 2025.

[28] D. R. K. Ports and K. Grittner, "Serializable snapshot isolation in PostgreSQL," *Proceedings of the VLDB Endowment*, vol. 5, no. 12, pp. 1850–1861, 2012.

[29] C. H. Papadimitriou, "The serializability of concurrent database updates," *J. ACM*, vol. 26, no. 4, pp. 631–653, 1979.

[30] N. Crooks, Y. Pu, L. Alvisi, and A. Clement, "Seeing is believing: A client-centric specification of database isolation," in *Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*, 2017, pp. 73–82.

[31] A. Cerone, G. Bernardi, and A. Gotsman, "A framework for transactional consistency models with atomic visibility," in *26th International Conference on Concurrency Theory (CONCUR)*, ser. LIPIcs, vol. 42, 2015, pp. 58–71.

[32] G. Bernardi and A. Gotsman, "Robustness against consistency models with atomic visibility," in *27th International Conference on Concurrency Theory (CONCUR)*, ser. LIPIcs, vol. 59, 2016, pp. 7:1–7:15.

[33] S. M. Beillahi, A. Bouajjani, and C. Enea, "Checking robustness against snapshot isolation," in *Proc. 31st Int. Conf. Computer Aided Verification (CAV)*, 2019.

[34] R. Biswas and C. Enea, "On the complexity of checking transactional consistency," *Proceedings of the ACM on Programming Languages*, vol. 3, no. OOPSLA, pp. 165:1–165:28, 2019.

[35] R. Guerraoui, M. Herlihy, and B. Pochon, "Toward a theory of transactional contention managers," in *Proceedings of the 24th ACM Symposium on Principles of Distributed Computing (PODC)*, 2005, pp. 258–264.

[36] W. N. Scherer III and M. L. Scott, "Advanced contention management for dynamic software transactional memory," in *Proceedings of the 24th ACM Symposium on Principles of Distributed Computing (PODC)*, 2005, pp. 240–248.

[37] A. Welc, B. Saha, and A.-R. Adl-Tabatabai, "Irrevocable transactions and their applications," in *Proc. 20th ACM Symp. Parallelism in Algorithms and Architectures (SPAA)*, 2008.

[38] C. Hawblitzel, J. Howell, M. Kapritsos, J. R. Lorch, B. Parno, M. L. Roberts, S. Setty, and B. Zill, "IronFleet: Proving practical distributed systems correct," in *Proceedings of the 25th ACM Symposium on Operating Systems Principles (SOSP)*, 2015, pp. 1–17.

[39] J. R. Wilcox, D. Woos, P. Panchekha, Z. Tatlock, X. Wang, M. D. Ernst, and T. Anderson, "Verdi: A framework for implementing and formally verifying distributed systems," in *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2015, pp. 357–368.

[40] M. Lesani, C. J. Bell, and A. Chlipala, "Chapar: Certified causally consistent distributed key-value stores," in *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2016.

[41] K. Kingsbury, "Jepsen: Distributed systems safety analysis," https://jepsen.io, 2013–present, accessed 2026.

[42] C. Tan, C. Zhao, S. Mu, and M. Walfish, "Cobra: Making transactional key-value stores verifiably serializable," in *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020, pp. 63–80.

[43] K. Kingsbury and P. Alvaro, "Elle: Inferring isolation anomalies from experimental observations," arXiv:2003.10554, 2020.

[44] K. Huang, S. Liu, Z. Chen, H. Wei, D. Basin, H. Li, and A. Pan, "Efficient black-box checking of snapshot isolation in databases," *Proc. VLDB Endow.*, vol. 16, no. 6, 2023.

[45] J. Zhang, Y. Ji, S. Mu, and C. Tan, "Viper: A fast snapshot isolation checker," in *Proc. 18th European Conf. Computer Systems (EuroSys)*, 2023.

[46] S. Liu, L. Gu, H. Wei, and D. Basin, "Plume: Efficient and complete black-box checking of weak isolation levels," *Proc. ACM Program. Lang.*, vol. 8, no. OOPSLA2, 2024.

[47] J. Clark, A. F. Donaldson, J. Wickerson, and M. Rigger, "Validating database system isolation level implementations with version certificate recovery," in *Proc. 19th European Conf. Computer Systems (EuroSys)*, 2024.

[48] S. Kim, S. Moon, R. Tabrizi, N. Lee, M. W. Mahoney, K. Keutzer, and A. Gholami, "An LLM compiler for parallel function calling," in *Proc. 41st Int. Conf. Machine Learning (ICML)*, 2024.

[49] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A benchmark for tool-agent-user interaction in real-world domains," arXiv:2406.12045, 2024.

[50] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.

[51] Temporal Technologies, "Temporal: durable execution platform," https://temporal.io, accessed 2026.

[52] S. Burckhardt, C. Gillum, D. Justo, K. Kallas, C. McMahon, and C. S. Meiklejohn, "Durable Functions: Semantics for stateful serverless," *Proceedings of the ACM on Programming Languages (OOPSLA)*, vol. 5, 2021.

[53] H. Lyu, D. Zhang, M. Wu, X. Wei, and H. Chen, "CoAgent: Concurrency control for multi-agent systems," arXiv preprint arXiv:2606.15376, 2026.

[54] M. Liu, T. Chen, Z. Xu, X. Jiang, and Y. Dong, "Multi-agent collaboration with state management," arXiv preprint arXiv:2605.20563, 2026.

[55] S. Khan, "S-Bus: Automatic read-set reconstruction for multi-agent LLM state coordination," arXiv preprint arXiv:2605.17076, 2026.

[56] W. Zhou, Z. Wang, Z. Peng, H. Chen, Y. Zhang, and G. Yu, "ATCC: Adaptive concurrency control for unforeseen agentic transactions," arXiv preprint arXiv:2603.13906, 2026.

[57] U. Çalıkyılmaz, N. Nayak, J. Groppe, and S. Groppe, "OptiMA: A transaction-based framework with throughput optimization for very complex multi-agent systems," arXiv preprint arXiv:2511.03761, 2025.

[58] V. Parakhin, "Token coherence: Adapting MESI cache protocols to minimize synchronization overhead in multi-agent LLM systems," arXiv preprint arXiv:2603.15183, 2026.

[59] S. G. Patil, T. Zhang, V. Fang, Noppapon C., R. Huang, A. Hao, M. Casado, J. E. Gonzalez, R. A. Popa, and I. Stoica, "GoEX: Perspectives and designs towards a runtime for autonomous LLM applications," arXiv preprint arXiv:2404.06921, 2024.

[60] M. Xavier, Vaisakh M. A., M. Jolly, and M. Xavier, "Agentproof: Static verification of agent workflow graphs," arXiv preprint arXiv:2603.20356, 2026.

[61] V. Hadzilacos, "A theory of reliability in database systems," *J. ACM*, vol. 35, no. 1, pp. 121–145, 1988.

[62] T. Kim, X. Wang, N. Zeldovich, and M. F. Kaashoek, "Intrusion recovery using selective re-execution," in *Proc. 9th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2010.

[63] R. Chandra, T. Kim, M. Shah, N. Narula, and N. Zeldovich, "Intrusion recovery for database-backed web applications," in *Proc. 23rd ACM Symp. Operating Systems Principles (SOSP)*, 2011.

[64] R. Chandra, T. Kim, and N. Zeldovich, "Asynchronous intrusion recovery for interconnected web services," in *Proc. 24th ACM Symp. Operating Systems Principles (SOSP)*, 2013.

[65] A. Skiadopoulos, Q. Li, P. Kraft, K. Kaffes, D. Hong, S. Mathew, D. Bestor, M. Cafarella, V. Gadepally, G. Graefe, J. Kepner, C. Kozyrakis, T. Kraska, M. Stonebraker, L. Suresh, and M. Zaharia, "DBOS: A DBMS-oriented operating system," *Proceedings of the VLDB Endowment*, vol. 15, no. 1, pp. 21–30, 2021.

[66] H. Zhang, A. Cardoza, P. B. Chen, S. Angel, and V. Liu, "Fault-tolerant and transactional stateful serverless workflows," in *Proc. 14th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2020.

[67] Z. Jia and E. Witchel, "Boki: Stateful serverless computing with shared logs," in *Proc. 28th ACM Symp. Operating Systems Principles (SOSP)*, 2021.

[68] S. Qi, X. Liu, and X. Jin, "Halfmoon: Log-optimal fault-tolerant stateful serverless computing," in *Proc. 29th ACM Symp. Operating Systems Principles (SOSP)*, 2023.

[69] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," *arXiv preprint arXiv:2310.08560*, 2023.

[70] A. Reuter and H. Wächter, "The ConTract model," *IEEE Data Engineering Bulletin*, vol. 14, no. 1, pp. 39–43, 1991.

[71] T. Chajed, J. Tassarotti, M. F. Kaashoek, and N. Zeldovich, "Verifying concurrent, crash-safe systems with Perennial," in *Proc. 27th ACM Symposium on Operating Systems Principles (SOSP)*, 2019, pp. 243–258.

[72] T. Chajed, J. Tassarotti, M. Theng, R. Jung, M. F. Kaashoek, and N. Zeldovich, "GoJournal: a verified, concurrent, crash-safe journaling system," in *Proc. 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2021, pp. 423–439.

[73] T. Chajed, J. Tassarotti, M. F. Kaashoek, and N. Zeldovich, "Argosy: verifying layered storage systems with recovery refinement," in *Proc. 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2019, pp. 1054–1068.

[74] K. Havelund and G. Roşu, "Synthesizing monitors for safety properties," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2002, pp. 342–356.

[75] M. Leucker and C. Schallhart, "A brief account of runtime verification," *Journal of Logic and Algebraic Programming*, vol. 78, no. 5, pp. 293–303, 2009.

[76] A. Bauer, M. Leucker, and C. Schallhart, "Runtime verification for LTL and TLTL," *ACM Transactions on Software Engineering and Methodology*, vol. 20, no. 4, pp. 14:1–14:64, 2011.

[77] T. Xu *et al.*, "Verification-aware planning for multi-agent systems," *arXiv preprint arXiv:2510.17109*, 2025.

[78] E. Fang, "AgentVerify: Compositional formal verification of AI agent safety properties via LTL model checking," Preprints.org, 2026, manuscript 202604.1029.

[79] A. Lomuscio, H. Qu, and F. Raimondi, "MCMAS: an open-source model checker for the verification of multi-agent systems," *International Journal on Software Tools for Technology Transfer*, vol. 19, no. 1, pp. 9–30, 2017.

[80] R. E. Strom and S. Yemini, "Optimistic recovery in distributed systems," *ACM Transactions on Computer Systems*, vol. 3, no. 3, pp. 204–226, 1985.

[81] E. N. Elnozahy, L. Alvisi, Y.-M. Wang, and D. B. Johnson, "A survey of rollback-recovery protocols in message-passing systems," *ACM Computing Surveys*, vol. 34, no. 3, pp. 375–408, 2002.

[82] Y. Zheng, Y. Yang, W. Zhang, and A. Quinn, "ACRFence: Preventing semantic rollback attacks in agent checkpoint-restore," arXiv preprint arXiv:2603.20625, 2026.

[83] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," arXiv preprint arXiv:2503.18666, 2025.

[84] W. Zhou, Y. Zhang, Z. Peng, and G. Yu, "TxnSails: Achieving serializable transaction scheduling with self-adaptive isolation level selection," arXiv preprint arXiv:2502.00991, 2025.

[85] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1993.

[86] M. Herlihy and E. Koskinen, "Transactional boosting: A methodology for highly-concurrent transactional objects," in *Proc. 13th ACM SIGPLAN Symp. Principles and Practice of Parallel Programming (PPoPP)*, 2008, pp. 207–216.

[87] T. Kraska, M. Hentschel, G. Alonso, and D. Kossmann, "Consistency rationing in the cloud: Pay only when it matters," *Proceedings of the VLDB Endowment*, vol. 2, no. 1, pp. 253–264, 2009.

[88] X. Li, Y. Wang, H. Lu, Z. Chen, M. Li, P. Song, M. Zheng, and T. Cai, "MemTX: Transactional belief commit for stateful agent memory," arXiv preprint arXiv:2607.23929, 2026.

[89] X. Yang, L. Li, Z. Ji, T. J. Zhang, and W. Jiang, "Position: Multi-agent systems should prioritize concurrency control," arXiv preprint arXiv:2608.18092, 2026.

[90] Y. Margalit, N. Cohen-Inger, E. Avram, R. Taig, and O. Margalit, "Governed shared memory for multi-agent LLM systems," arXiv preprint arXiv:2606.24535, 2026.

[91] T. E. Kim, X. He, D. Jain, A. Agrawal, N. Arabzadeh, and F. Diaz, "Multi-agent transactive memory," arXiv preprint arXiv:2606.19911, 2026.

[92] I. Gim, S. Lee, and L. Zhong, "Asynchronous LLM function calling," arXiv preprint arXiv:2412.07017, 2024.

[93] G. Feng, H. Mao, P. Dutta, and J. E. Gonzalez, "Concurrency without model changes: Future-based asynchronous function calling for LLMs," arXiv preprint arXiv:2605.15077, 2026.

[94] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, "Identifying the risks of LM agents with an LM-emulated sandbox," in *Int. Conf. Learning Representations (ICLR)*, 2024.

[95] T. Shi, J. He, Z. Wang, L. Wu, H. Li, W. Guo, and D. Song, "Progent: Programmable privilege control for LLM agents," arXiv preprint arXiv:2504.11703, 2025.

[96] Y. Wu, F. Roesner, T. Kohno, N. Zhang, and U. Iqbal, "IsolateGPT: An execution isolation architecture for LLM-based agentic systems," in *Network and Distributed System Security Symp. (NDSS)*, 2025.

[97] I. Santos-Grueiro, "Temporary authority, permanent effects: Commit-time authorization for LLM agents," arXiv preprint arXiv:2607.10487, 2026.

[98] Z. Zheng, J. Long, B. Li, and J. Yao, "Stale does not mean unsafe: Guard precision for tool-using LLM agents under infrastructure state races," arXiv preprint arXiv:2609.29522, 2026.

[99] H. Zhang, H. Zhang, Z. Liang, Y. Yan, D. Zuo, and H. Wang, "Beyond approved actions: Runtime validation of persistent outcomes in agent workflows," arXiv preprint arXiv:2609.31301, 2026.

[100] Y. Zhang, R. Sharma, N. Vegesna, Z. Fu, C. Liu, and T. Krishnamurthy, "Serving a revisable world: Versioned execution for interruptible agents," arXiv preprint arXiv:2610.01160, 2026.

[101] L. Fan, Z. Li, Y. Tian, Y. Wang, R. Li, and X. Wang, "The granularity mismatch in agent security: Argument-level provenance solves enforcement and isolates the LLM reasoning bottleneck," arXiv preprint arXiv:2605.11039, 2026.

[102] P. Bailis, A. Davidson, A. Fekete, A. Ghodsi, J. M. Hellerstein, and I. Stoica, "Highly available transactions: Virtues and limitations," *Proc. VLDB Endow.*, vol. 7, no. 3, pp. 181–192, 2014.

[103] W. E. Weihl, "Commutativity-based concurrency control for abstract data types," *IEEE Trans. Comput.*, vol. 37, no. 12, pp. 1488–1505, 1988.

[104] D. Shasha, F. Llirbat, E. Simon, and P. Valduriez, "Transaction chopping: Algorithms and performance studies," *ACM Trans. Database Syst.*, vol. 20, no. 3, pp. 325–363, 1995.

[105] P. Buneman, S. Khanna, and W.-C. Tan, "Why and where: A characterization of data provenance," in *Proc. ICDT*, 2001, pp. 316–330.

[106] T. J. Green, G. Karvounarakis, and V. Tannen, "Provenance semirings," in *Proc. PODS*, 2007, pp. 31–40.

[107] A. Gupta and I. S. Mumick, "Maintenance of materialized views: Problems, techniques, and applications," *IEEE Data Eng. Bull.*, vol. 18, no. 2, pp. 3–18, 1995.

[108] T. Akidau, A. Balikov, K. Bekiroğlu, S. Chernyak, J. Haberman, R. Lax, S. McVeety, D. Mills, P. Nordstrom, and S. Whittle, "MillWheel: Fault-tolerant stream processing at internet scale," *Proc. VLDB Endow.*, vol. 6, no. 11, pp. 1033–1044, 2013.

[109] P. Carbone, S. Ewen, G. Fóra, S. Haridi, S. Richter, and K. Tzoumas, "State management in Apache Flink: Consistent stateful distributed stream processing," *Proc. VLDB Endow.*, vol. 10, no. 12, pp. 1718–1729, 2017.

[110] B. Atil *et al.*, "LLM stability: A detailed analysis with some surprises," arXiv preprint arXiv:2408.04667, 2024.

[111] Z. Sun, X. Wang, and G. Li, "Agentic transaction: Towards ACID-compliant agent systems," arXiv preprint arXiv:2608.13900, 2026.

[112] LangGraph 1.2, Letta, CrewAI 1.15, GitHub repositories, 2026.